

Alset IoT Hugs the Lanes

Developed By Team Project Larry



Table of Contents

Table of Contents.....	2
1. Introduction.....	5
1.1 Purpose.....	5
1.2 Project Scope.....	5
1.3 Features.....	5
1.4 Team Qualifications.....	6
1.5 Development Process.....	6
2. Architecture.....	7
2.1 Localization.....	7
2.2 Perception.....	7
2.3 Planning.....	8
2.4 Vehicle Control.....	8
2.5 System Management.....	8
2.6 Sensor Fusion.....	9
2.7 Display.....	9
3. Requirements.....	9
3.1 Functional Requirements.....	9
3.1.2 Automatic Lane Changing.....	10
3.1.3 Cruise Control.....	10
3.1.4 Dangerous Merge Prevention.....	11
3.1.5 Hazard Lights.....	11
3.1.7 Automatic Light System Activation.....	12
3.1.8 Automatic Windshield Wiper Activation.....	12
3.1.9 Temperature Control.....	13
3.1.10 Driver Input to Override Autonomous Driving System.....	13
3.2 Nonfunctional Requirements.....	14
4. Requirements Modeling.....	16
4.1 Use Case Scenarios.....	16
4.1.1 Sensors Detect an Object in the Path of the Vehicle.....	17
4.1.2 Driver Activates Automatic Lane Change While in Cruise Control.....	17
4.1.3 Driver Engages Brakes While in Cruise Control.....	18
4.1.4 Light Sensor Activates Headlights.....	18
4.1.5 Technician Requests Log Files.....	19
4.2 Activity Diagrams.....	20
4.2.1 Sensors Detect an Object in the Path of the Vehicle.....	20
4.2.2 Driver Activates Automatic Lane Change While in Cruise Control.....	21
4.2.3 Driver Engages Brakes While in Cruise Control.....	22
4.2.4 Light Sensor Activates Headlights.....	22

4.2.5 Technician Requests Log Files.....	23
4.3 Sequence Diagrams.....	24
4.3.1 Sensors Detect an Object in the Path of the Vehicle.....	24
4.3.2 Driver Activates Automatic Lane Change While in Cruise Control.....	24
4.3.3 Driver Engages Brakes While in Cruise Control.....	25
4.3.4 Light Sensor Activates Headlights.....	25
4.3.5 Technician Requests Log Files.....	26
4.4 Classes.....	27
4.5 State Diagrams.....	28
4.5.1 Sensors detect object(s) in path of vehicle.....	28
4.5.2 Driver requests automatic lane change.....	28
4.5.3 Driver Engages Brakes While in Cruise Control.....	29
4.5.4 Light Sensors Activate Headlights.....	29
4.5.5 Technician Requests Log Files.....	29
5. Design.....	30
5.1 Software Architecture.....	30
5.1.1 Data Centered Architecture.....	30
5.1.2 Data Flow Architecture.....	31
5.1.3 Call Return Architecture.....	32
5.1.4 Object-Oriented Architecture.....	33
5.1.5 Layered Architecture.....	34
5.1.6 Model View Controller Architecture.....	35
5.1.7 Finite State Machine Architecture.....	36
5.1.8 Chosen Architecture.....	36
5.2 Interface Design.....	37
5.2.1 Data Aggregation Interface.....	37
5.2.2 Planning-VCS Interface.....	37
5.2.3 Interaction Interface.....	37
5.3 Component-Level Design.....	38
5.3.1 Data Aggregation Interface.....	38
5.3.2 Planning-VCS Interface.....	42
5.3.3 Interaction Interface.....	48
6. Project Code.....	50
6.1 Data Aggregation.....	50
6.2 Planning-VCS.....	56
6.3 Interaction.....	62
6.3.1 Main.....	70
7.1 Scenario-Based Testing.....	74
7.1.1 Sensors Detect an Emergency break.....	74
7.1.2 Driver Activates Automatic Lane Change While in Cruise Control.....	75
7.1.3 Driver Engages Brakes While in Cruise Control.....	75

7.1.4 Light Sensor Activates Headlights.....	75
7.1.5 Technician Requests Log Files.....	76
7.2 Validation Testing.....	77
7.2.1 Adaptive Braking.....	77
7.2.2 Automatic Lane Changing.....	77
7.2.3 Cruise Control.....	77
7.2.4 Dangerous Merge Prevention.....	78
7.2.5 Hazard Lights.....	78
7.2.6 Automatic Light System Activation.....	78
7.2.8 Automatic Windshield Wiper Activation.....	78
7.2.9 Temperature control.....	79
7.2.10 Driver Input to Override Autonomous Driving System.....	79

1. Introduction

1.1 Purpose

This document will outline the foundation of the “Alset IoT Hugs the Lanes” project, an IoT-enabled self-driving system. Given the frequency of car crashes, fatal and otherwise, it is imperative to implement a system that will use modern technology to mitigate such accidents. Many car accidents are caused by human errors, such as distracted or impaired driving, or fatigue. This autonomous system will gather IoT sensor data to significantly reduce these errors and limit driver-related accidents. An IoT sensor technology based architecture is the most advanced architecture for such systems and is critical to its success and functionality. To utilize it, this system will serve as an onboard analytics engine (IoT Engine) used to analyze information and make decisions that are then communicated to the Car Control System (CCS) and will ultimately provide the car with self driving and assisted driving capabilities in order to maximize the safety of passengers. As this system is heavily dependent on the accurate collection of data from the IoT sensor network, various sensors will be placed in strategic locations on the car to collect critical data that will be sent to the IoT Engine. This data will also be transmitted to the cloud to provide insight for future updates and to continuously improve the system. This self-driving software is a real-time system designed to improve road safety with continuous updates from developers and technicians while prioritizing user comfort and safety.

1.2 Project Scope

The first phase of this project will showcase the system’s ability to gather information from the IoT sensors and send them to the Car Control System. The CCS will draw basic conclusions from this information and produce simple warnings to the driver, but will not affect the car’s movement. The objective of this phase is to construct a working interface for sensor communication to have a solid foundation to utilize and build from in future, more complex phases. The overall scope of this software development project is to develop a centralized self-driving vehicle control system integrated with communication protocols for data exchange, and an array of sensors for real-time environmental perception. This scope explicitly excludes hardware manufacturing, physical vehicle modifications, and any non-essential features beyond autonomous driving functionalities.

1.3 Features

This project plans to implement several features including self-driving and assisted driving capabilities, a command system, and cloud connectivity. Such attributes include lane assist, blindspot detection and automatic braking systems. The command system will allow the driver to input various commands and receive the status of the Car Control System. Similarly, the cloud connectivity feature will enable the system to utilize remote cloud servers, rather than relying solely on the infrastructure, to stay up to date and send feedback to developers and technicians. An interface will be provided for technicians and developers to constructively tap in and maintain the software. In order to ensure safety and reliability, the system will implement various security measures to prevent malicious attacks, in place with a number of failsafes. The system will regularly receive updates and notify users when it should be taken in for service.

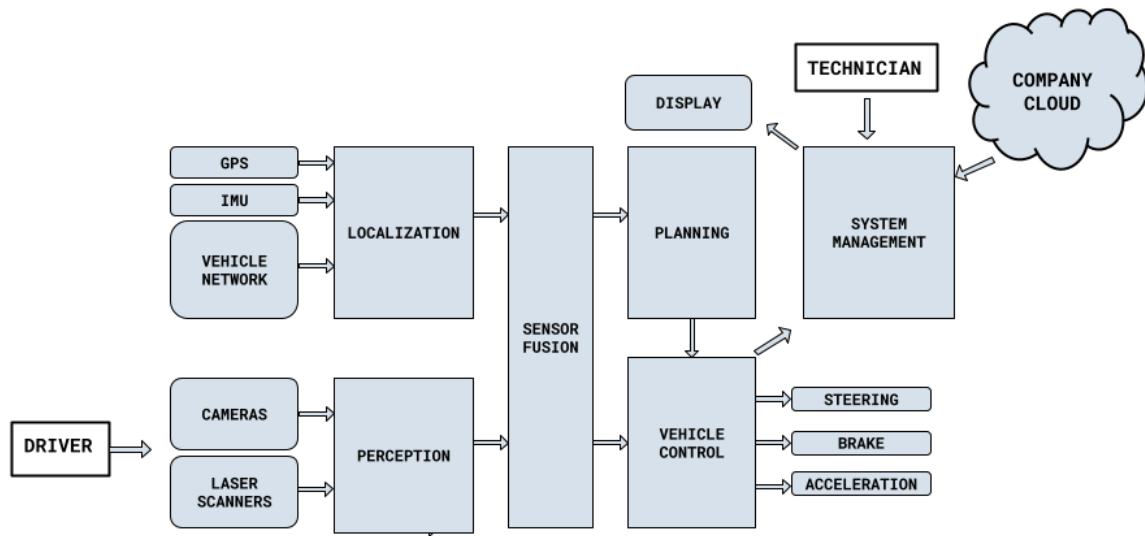
1.4 Team Qualifications

Team Project Larry is a tech-driven development team that utilizes agile software development processes to build safe and efficient systems to satisfy user safety and needs. This team consists of four, second-year, computer science students enrolled at Stevens Institute of Technology. Each team member contributes a unique skillset to the team, fostering a collective strength in communication, problem-solving approaches, and project management strategies. This team operates collaboratively and maintains communication and organization throughout all phases of project development.

1.5 Development Process

Team Project Larry utilizes a waterfall model approach to define a system project outline and obtain a predictable process workflow. This model is a sequential and linear approach to software development, where each phase is to be completed before moving forward. These phases focus on planning, modeling, construction, and deployment of the IoT Engine in a disciplined, organized framework. Although the waterfall model is the oldest paradigm for software engineering, this structure allows the team to establish the driver's general and safety requirements early on to ensure that the system prioritizes the necessary features for future phases of development. Given the complexity and safety-critical nature of this self-driving system, this process model aligns with the team's goals to emphasize safety and reliability of system components and communication.

2. Architecture



The model above illustrates the system architecture that the vehicle will maintain to operate as a self-driving car. A collection of data from various sensors and devices will be fed to the localization and perception modules for the vehicle to understand its surroundings to generate its behavior and trajectory. This data will be processed and analyzed in the planning module, and passed to the vehicle control system to handle the car's functionality and movement while working concurrently with the physical components of the vehicle. All of this data will be sent to the company cloud to be stored. The driver will not have insight to the vehicle control system, but will be guided by the perception module. Similarly, technicians will have access to system management to warrant software updates and obtain significant vehicle information for repairs. This architecture is built to maintain the vehicle's safety and reliability in real-time while operating in an unpredictable, ever-changing environment.

2.1 Localization

The system uses several technologies for precise and reliable localization. These are including but not limited to: surround sensors, satellite navigation and inertial sensors. Recorded data is collected from the vehicle's surround sensors to create an independent map layer which depicts the unique features of the road. The fusion of data from radar and camera sensors provide the system with important information on objects in the vehicle's surroundings. In order to meet the strict safety requirements of highly automated driving, surround sensors and the road signature are coupled with advanced satellite-based localization. Together, hardware and software serve as a system to determine the vehicle's position.

2.2 Perception

The perception module will focus on gathering data from the surrounding environment to assist the vehicle in planning the path ahead. The system will accumulate data from cameras, laser scanners, radar sensors and various others, to build a detailed representation of the proximate and distant world around the vehicle. The information provided by these devices will be processed and analyzed to aid the system

in making sudden, navigational decisions that are imminent to the safety and reliability of the vehicle. The system must be able to adapt to changing environments, including varied road and weather conditions, parking lots, construction zones, urban communities, and many more. This module will also contribute to the vehicle's ability to identify and anticipate obstacles like road signs, other vehicles, and pedestrians in real-time. The perception system will work concurrently with localization elements to provide critical operative functionalities to the vehicle.

2.3 Planning

The planning module will work with the given data from perception and localization systems in order to make decisions about the vehicle's behavior and trajectories. The sensors and devices used in perception and localization will generate a dynamic and elaborate representation of the vehicle's surroundings for the planning system to use in order to plan routes and predict behaviors of other road users. This module will focus on creating a safe and efficient route for the vehicle to travel from one destination to another, whilst maintaining safety, comfort and driving legality. This system will account for sudden and changing elements such as traffic conditions, lane changes, intersection crossings and unexpected obstacles. The main functionalities of the planning module consist of mission planning, behavior planning and trajectory generation with priorities for vehicle safety, reliability and navigation.

2.4 Vehicle Control

The vehicle control module of the system will handle all functionality that will control the car's movement. It synthesizes the vehicle's plans with conclusions drawn by the sensor fusion module in order to make precise judgements to the vehicle's motion in the form of steering, braking, and acceleration control.

2.5 System Management

The system management module will be responsible for the system's configuration, maintenance, and monitoring. The system's features will be configurable by the user and technician. As such, an interface for system management is necessary. From the dashboard, the user will be able to do the following:

- Toggle warnings
- *Implement custom warnings*
- Toggle assisted and autonomous driving capabilities

In addition to the user features, technicians will have the capability to:

- Retrieve system logs
- Monitor resource usage and allocation
- Configure security settings
- Run diagnostics

The system management module will also be responsible for fault detection and will continuously monitor the performance of safety-critical functions and initiate fail-safe mechanisms in case of emergencies.

2.6 Sensor Fusion

The sensor fusion module is crucial for integrating data from radar, cameras, and inertial sensors, providing a comprehensive view of the vehicle's surroundings. It enables accurate decision-making by refining vehicle positioning through the Localization module and enhancing object detection via the Perception module. This ensures the Vehicle Control module receives reliable data for effective navigation and response to road conditions. Key functions include:

- ⇒ Unified environmental modeling from diverse sensor inputs.
- ⇒ Improved localization and object detection accuracy.
- ⇒ Reliable data provision for informed vehicle control.
- ⇒ Technicians can monitor sensor integration performance, adjust fusion parameters, and conduct system diagnostics, ensuring continuous operational safety and precision in the vehicle's maneuvers.

2.7 Display

The display module will assist in outputting necessary information to the driver in an aesthetic manner. The display module will feature an interactive display within, allowing for drivers to input commands while cruise control is activated. The display module will help users perform vehicle actions, access authorized data and receive information about the vehicle status.

3. Requirements

3.1 Functional Requirements

3.1.1 Emergency Braking

When the vehicle exceeds 125 miles per hour, which is the speed limit for this vehicle, the system will automatically initiate brakes to rapidly decrease the vehicle's speed to ensure safety.

Preconditions:

- System is activated.
- Vehicle is in motion.
- Vehicle speed exceeds 125 miles per hour.

Postconditions:

Vehicle speed is rapidly reduced to a safer speed below 125 miles per hour.

Process:

- 3.1.1. Speed sensors continuously monitor the vehicle's speed.
- 3.1.2. If the vehicle's speed exceeds 125 miles per hour, the sensor fusion module receives this data.
- 3.1.3. The sensor fusion module confirms the high speed and communicates it to the planning module.
- 3.1.4. The planning module sends a command to the vehicle control module to start emergency braking.
- 3.1.5. The vehicle control module activates the brakes, rapidly reducing the vehicle's speed.

3.1.6. The system logs the event in the system management module and an alert is displayed on the vehicle's dashboard indicating that emergency braking has been activated.

3.1.2 Automatic Lane Changing

The system will be able to automatically change lanes when the driver inputs a command to do so.

Preconditions:

- System is activated
- Vehicle is in motion
- Vehicle is on a marked road with multiple lanes
- Target lane is clear of any objects or obstructions and is a valid lane

Postconditions:

- Vehicle moves left or right in order to enter the target lane

3.1.2.1 Driver inputs a command to change lanes to a specified target lane (left or right)

3.1.2.2 GPS sends data to the localization module on whether or not the road is marked and if it has multiple lanes.

3.1.2.3 Cameras and laser scanners detect whether or not there are any obstructions to the target lane and forward the collected data to the perception module.

3.1.2.4 The sensor fusion module normalizes the data from the localization and perception modules and forwards it to the planning module.

3.1.2.5 The planning module confirms that the road is marked, has multiple lanes, and that the target lane is valid. If all of these conditions are not met, the command is terminated and a notification that it was terminated is sent to the display. Otherwise, the conclusions of the planning module are sent to the vehicle control module.

3.1.1.6 The vehicle control module steers the wheel left or right in order to enter the target lane and then maintains the vehicle's position once in said target lane.

3.1.1.7 The use of or failure to use automatic lane changing is added to the logs of the system management module.

3.1.3 Cruise Control

The system will have cruise control capabilities that will keep the vehicle centered in its lane and moving at a satisfactory speed. The vehicle will be able to accelerate or brake as needed in order to maintain these conditions.

Preconditions:

- System is activated
- Vehicle is in motion
- Vehicle is on a road with more than one lane, a speed limit of at least 25 mph, and few traffic lights or stop signs

Postconditions:

- Vehicle maintains its position in its current lane
- Vehicle accelerates or brakes in order to maintain a constant speed

3.1.3.1 Driver inputs a command to activate cruise control.

3.1.3.2 GPS forwards data about the current road to the localization module.

3.1.3.3 The sensor fusion module normalizes the GPS data and sends it to the planning module.

3.1.3.4 If the road does not meet the specified criteria to activate cruise control, terminate the command and send a notification via the display. Otherwise, cruise control is activated and the planning module reads real-time information from the sensor fusion.

3.1.3.5 The vehicle control module reads a stream of data from the planning module in order to make decisions on what appropriate actions to take, such as steering, acceleration, or braking.

3.1.3.6 If the conditions to activate cruise control are no longer met or the user inputs a command to deactivate cruise control, notify the user and deactivate cruise control.

3.1.4 Dangerous Merge Prevention

The cameras and laser scanners will continuously scan the vehicle's surroundings to detect objects that may occupy the vehicle's path. If the driver tries to merge into one of these objects or something that may occupy the same space as the vehicle imminently, the system will prevent the driver from doing so.

Preconditions:

- System is activated
- Vehicle is in motion
- Object of significant size is detected occupying the space the driver is merging into

Postconditions:

- Vehicle steers in the opposite direction, preventing the merge

3.1.4.1 Driver begins to merge to the left or right

3.1.4.2 Laser scanners and cameras detect an object of significant size currently occupying the space that the vehicle is merging into or will be soon. This data is sent to the perception module.

3.1.4.3 The sensor fusion module normalizes the data from the perception module and forwards it to the planning module.

3.1.4.4 The planning module concludes that the driver is trying to merge into an object and forwards data to the vehicle control module.

3.1.4.5 The vehicle control system steers the wheel in the opposite direction that the driver is trying to merge, keeping it straight and preventing the merge.

3.1.4.6 A notification is shown on the display that a dangerous merge was prevented.

3.1.5 Hazard Lights

The system will allow the user to input to turn on hazard lights in case of an emergency. The vehicle will display that the lights are on

Preconditions:

- System is activated
- Vehicle is in on
- User turns on or off the hazard lights

Postconditions:

- The light turns off or on.
- The correct warning is displayed.

3.1.5.1 The vehicle control system activates the warning system, which in turn activates the hazard lights.

3.1.7 Automatic Light System Activation

The system will have specific light, camera and temperature sensors to detect and activate the system when the vehicle needs to utilize fog lights, headlights or high beams.

Preconditions:

- System is activated
- Vehicle is in motion
- Visibility issues are detected

Postconditions:

- The vehicle's fog lights, headlights, and/or high beams are activated
- The light system is deactivated once visibility is restored, or the vehicle is powered off

3.1.7.1.1 The vehicle control system activates windshield wipers.

3.1.7.1.2 Illuminance sensors installed near the dashboard are used to measure the levels of light that are detected. This data is normalized by Sensor Fusion and sent to the Planning module to be analyzed. If the level drops below a certain threshold, Planning will alert the VCS.

3.1.7.1.3 The vehicle control system activates headlights.

3.1.7.2.1 Visibility sensors located near the front of the vehicle are used to gauge how well the driver can see. This data is constantly collected, fused and sent to the Planning module. If the visibility level drops below a certain threshold, Planning will alert the VCS.

3.1.7.2.2 The vehicle control system activates fog lights.

3.1.7.3.1 Cameras that are centrally mounted behind the windshield of the vehicle will continuously collect data with regards to the front projection of the vehicle.

3.1.7.3.2 This data is normalized by Sensor Fusion and sent to the Planning module to be analyzed. If the ambient light level drops below a certain threshold, traffic in the front of the vehicle is analyzed in the by the Planning module.

3.1.7.3.3 If no traffic is detected in a large range in front of the vehicle, the Planning module will alert the VCS.

3.1.7.3.4 The vehicle control system activates the high beams.

3.1.8 Automatic Windshield Wiper Activation

The system will detect rain and automatically activate the windshield wipers at an appropriate rate to ensure that the driver has visibility in adverse weather conditions.

Preconditions:

- System is activated
- Windshield wipers are set to automatic
- Sufficient amount of rain has been detected on the windshield (3 drops per square inch per second for at least 5 seconds)

Postconditions:

- Windshield wipers are activated until rain is no longer detected on the windshield

3.1.8.1 Sensors on the windshield detect a sufficient amount of rain and forward the data to the perception module.

3.1.8.2 Data is normalized by the sensor fusion module and sent to the planning module.

3.1.8.3 The planning module concludes that there is a significant amount of rainfall and sends data to the vehicle control system.

3.1.8.4 The vehicle control system activates the windshield wipers.

3.1.9 Temperature Control

The system will maintain the interior temperature of the vehicle within a specified range based on the driver's input.

Preconditions:

- System is activated
- Vehicle is in motion or stationary.
- Driver specifies desired interior temperature through the vehicle's climate control interface.

Postconditions:

- Interior temperature of the vehicle is adjusted to the specified range.

3.1.9.1 Driver inputs desired interior temperature through the climate control interface.

3.1.9.2 The climate control system adjusts the heating or cooling settings to bring the interior temperature within the specified range.

3.1.9.3 If the driver changes the desired temperature, the process repeats to adjust the climate control settings accordingly

3.1.10 Driver Input to Override Autonomous Driving System

The system will allow the driver to place its own driving inputs, such as steering, accelerating or braking, to override the vehicle's autonomous driving system.

Preconditions:

- System is activated
- Vehicle is in motion
- Warning system is activated
- Driver places input, attempting to override the system

Postconditions:

- The vehicle's warning system is overridden
- The vehicle carries out driver input

3.1.10.1 The driver attempts to override the system by steering, accelerating or braking. The Vehicle Control System is alerted.

3.1.10.2 The system is overridden and the driver is in control of the vehicle. All systems remain active and the warning system is notified that the driver is overriding autonomous features.

3.1.10.3 The warning system audibly alerts the driver with intensifying beeps to convey that the driver is overriding the system.

3.1.10.4 The perception module will continue to collect data from sensors, cameras and scanners to fuse to the VCS to monitor the driver's actions and vehicle's performance.

3.1.10.5 The system is prepared to provide additional alerts if any issues are recognized.

3.1.10.6 The system is ready to operate autonomously again whenever the driver indicates with the push of a specific button.

3.2 Nonfunctional Requirements

3.2.1 System Updates

The vehicle's software system will regularly receive updates to enhance functionality, introduce new features, and improve system security. Updates can be performed over-the-air (OTA), allowing for seamless integration without the need for physical dealership visits. Key aspects include:

3.2.1.1 Automatic Update Notifications:

The vehicle will notify the user of available updates through the display interface, ensuring the user is always aware of the latest enhancements.

3.2.1.2 Scheduled Updates:

Users can schedule updates for convenient times, such as overnight, to minimize disruption to vehicle use.

3.2.1.3 Update History Log:

The system will maintain a log of all performed updates, accessible through the display for user review.

3.2.2 Display

This includes vehicle status updates, lane switching, and other basic functionalities. The following is a list of some of the features that will be implemented as part of the display:

3.2.2.1 Driver Dashboard

The interactive vehicle dashboard will display a variety of essential vehicle data values. This includes headlight, turn signal, wiper, gear, cruise control and hazard lights status. The display will also feature the vehicle's current lane and interior temperature.

3.2.2.2 Perform Actions

The driver will be able to perform necessary actions like: braking, accelerating, changing gears/lights/wipers/turn signals or lanes, and activating/deactivating cruise control. vehicle status updates will also print to the driver; for example, if the vehicle is unable to switch lanes.

3.2.2.3 Logs and Data

The technician interface will have access to get the vehicle status for all data values, outputted aesthetically. It will also be able to access all of the logs from the vehicle's current operations. Other (unauthorized) users will be able to read data from the vehicle (more raw data, not formatted).

3.2.3 Performance

This section outlines the vehicle's performance capabilities, emphasizing efficiency, power, and handling. Performance metrics include:

3.2.3.1 Efficiency Metrics:

Information on the vehicle's speed and status throughout a trip, also providing insights into the vehicle's environment.

3.2.3.2 Power Output:

Data on vehicle movement, steering and acceleration capabilities, showcasing the vehicle's engineering excellence.

3.2.3.3 Handling Characteristics:

Details on the vehicle's system, braking performance, and steering responsiveness ensure a safe and enjoyable driving experience.

3.2.4 Security/Safety

Vehicle security is paramount, incorporating both physical and digital protection mechanisms. Security features encompass:

3.2.4.1 Data Safety:

All vehicle communications and data storage are safeguarded for user privacy. Sensitive vehicle information requires authorized access.

3.2.4.2 Hazard Detection System:

The vehicle is equipped with a real-time monitoring system to detect and alert users to potential hazards. Vehicle will have certain restrictions to ensure that gears and lanes are switched properly and safely, and that cruise control is activated only in safe conditions.

3.2.5 Reliability

Reliability ensures the vehicle consistently performs as expected, offering dependability and peace of mind to users. Reliability features include:

3.2.5.1 Diagnostics and Monitoring:

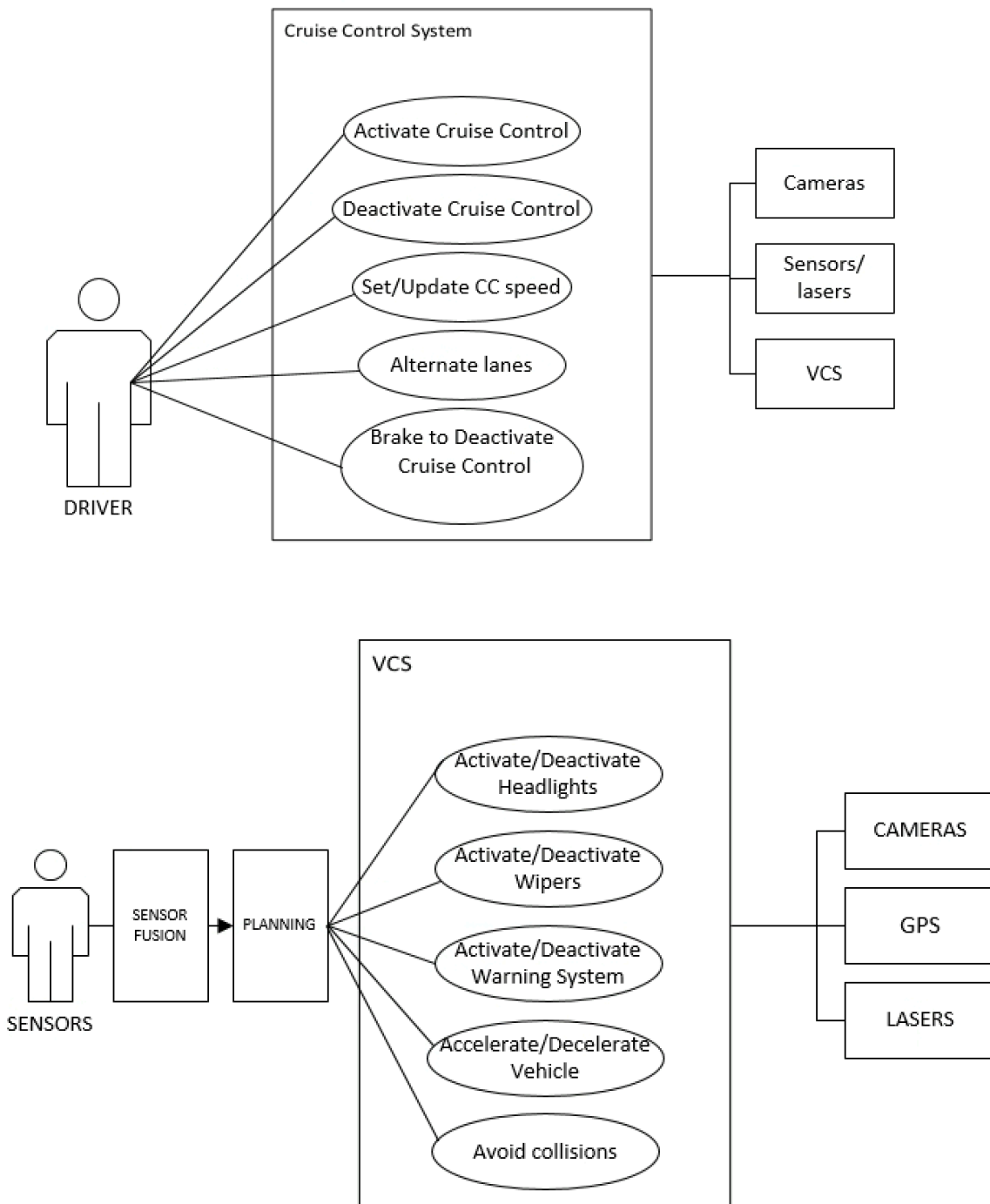
The vehicle constantly monitors its systems for any signs of malfunction, alerting the user to potential issues and hazards.

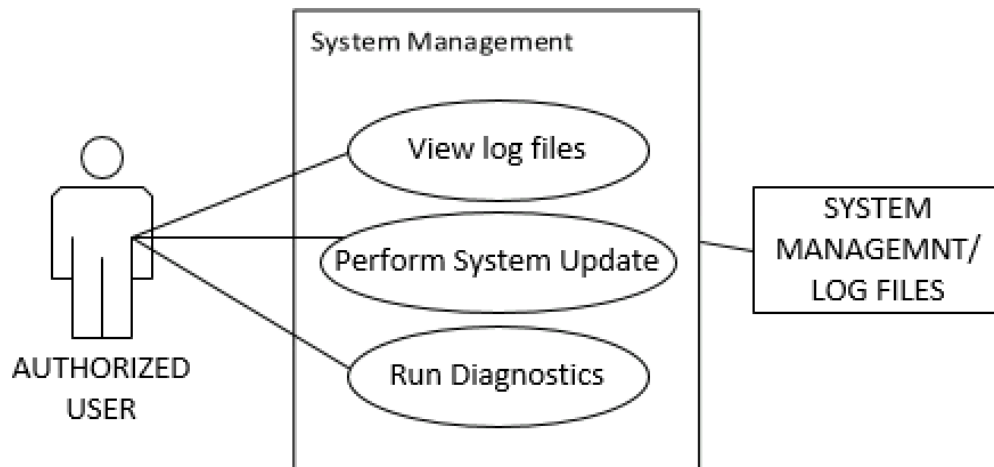
3.2.5.2 Redundancy Systems:

Critical systems have built-in redundancies, such as backup sensors and alternative power sources, to ensure the vehicle remains operational even if a primary system fails.

4. Requirements Modeling

4.1 Use Case Scenarios





4.1.1 Sensors Detect an Object in the Path of the Vehicle

Preconditions:

- Vehicle is powered on
- Vehicle is in motion
- All systems are active
- Sensor detects an object in the immediate path of the vehicle

Postconditions:

- Warning system activated
 - Display shows a warning
- The vehicle path is no longer obstructed

Trigger: Sensors detect an object in the immediate path of the vehicle

Steps:

1. Sensors detect an object in the path of the vehicle and sends the data to sensor fusion.
2. Sensor Fusion normalizes data and determines if the vehicle is on track to collide with the object and sends data to planning.
3. Planning activates the Warning System and sends data to the Display module.
4. Planning module determines how hard to apply the brakes and forwards the data to the Vehicle Control System.
5. The Vehicle Control System applies the brakes at the intensity specified.
6. The event is logged by the System Management module.

Exceptions:

1. The object is no longer in the immediate path of the vehicle, so the brakes are not applied.
2. The object or vehicle changes trajectory, so the sensor data is normalized and reevaluated and the planning module calculates a new course of action.

4.1.2 Driver Activates Automatic Lane Change While in Cruise Control

Preconditions:

- Vehicle is powered on
- All systems are active
- Vehicle is in Cruise Control motion
- Driver inputs command for a lane change

Postconditions:

- Vehicle switches to requested lane
- Display shows a notification that the lane change was either successful or failed

Trigger: Driver engages turn signal

Steps:

1. Driver activates turn signal while in Cruise Control.
2. VCS receives request and sends data to the Planning module.
3. Sensors gather data from perception to be normalized in Sensor Fusion.
4. Sensor Fusion sends fused data to the Planning module.
5. The Planning module interprets data to determine if the lane change is safe and possible.
6. Planning commands the VCS to shift lanes.
7. The event is logged by the System Management module.

Exceptions:

1. Planning module determines that the lane switch is not safe and activates the Warning System to alert and display the issue to the driver and disengage the turn signal.

4.1.3 Driver Engages Brakes While in Cruise Control

Preconditions:

- Vehicle is powered on
- Vehicle is in Cruise Control motion
- All systems are active
- Driver engages brake pedal

Postconditions:

- Vehicle deactivates Cruise Control and returns to normal state
- The driver's display is updated

Trigger: Driver engages the brake pedal while in Cruise Control

Steps:

1. Driver presses the brake pedal while the vehicle is in Cruise Control mode.
2. The Vehicle Control System receives data that the brake has been activated by the Driver.
3. The VCS deactivates Cruise Control and sends this data to Planning.
4. Planning sends a request to the Display module to turn off the Cruise Control icon.
5. The event is logged by the System Management module.

Exceptions:

1. The brake is activated at the same time as the gas pedal. The Cruise Control remains on. The VCS sends data to Planning which activates the Warning System and requests for the Display module to output a warning and an audible beep to alert the driver and advise against doing so.

4.1.4 Light Sensor Activates Headlights

Preconditions:

- Vehicle is powered on
- All systems are active
- Vehicle is in a low visibility environment

Postconditions:

- Headlights are activated
- High beams are activated if necessary

Trigger: Light sensors detect a low light level.

Steps:

1. Light sensor continuously detects the amount of light in the external environment and sends data through Sensor Fusion to the Planning module.
2. Planning module receives the data and determines the light level is insufficient.
3. Planning sends a command to the Vehicle Control System to activate the headlights.
 - a. Using the given data, Planning determines whether or not to activate the high beams.
4. Planning sends commands to the Display module to show the headlight status.
 - a. Planning sends a command to Display for the highbeam status as well, if necessary.
5. The event is logged by the System Management module.

Exceptions:

1. One of the headlights is defective. Planning will receive this normalized data and activate the Warning System to request the Display module to output the defective headlight status.

4.1.5 Technician Requests Log Files

Precondition:

- Vehicle is powered on
- All systems are active
- Technician/Admin requests information about the vehicle

Postcondition:

- Technician/Admin accesses system status and log files
- System status is outputted to the display

Trigger: An authorized user requests vehicle information

Steps:

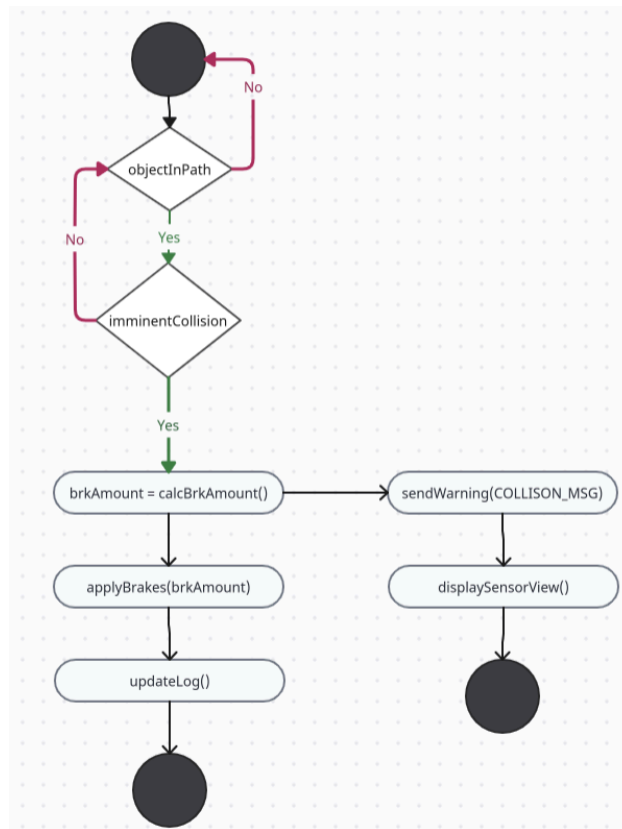
1. Display module prompts the user to sign in and sends data to Planning to validate credentials.
2. The display prompts the user to select a module to access logs from (will keep prompting until the user logs out).
3. Display sends a request for data to System Management.
4. System Management sends the Display module the necessary to display the requested module logs.
5. The event is logged by the System Management module.

Exceptions:

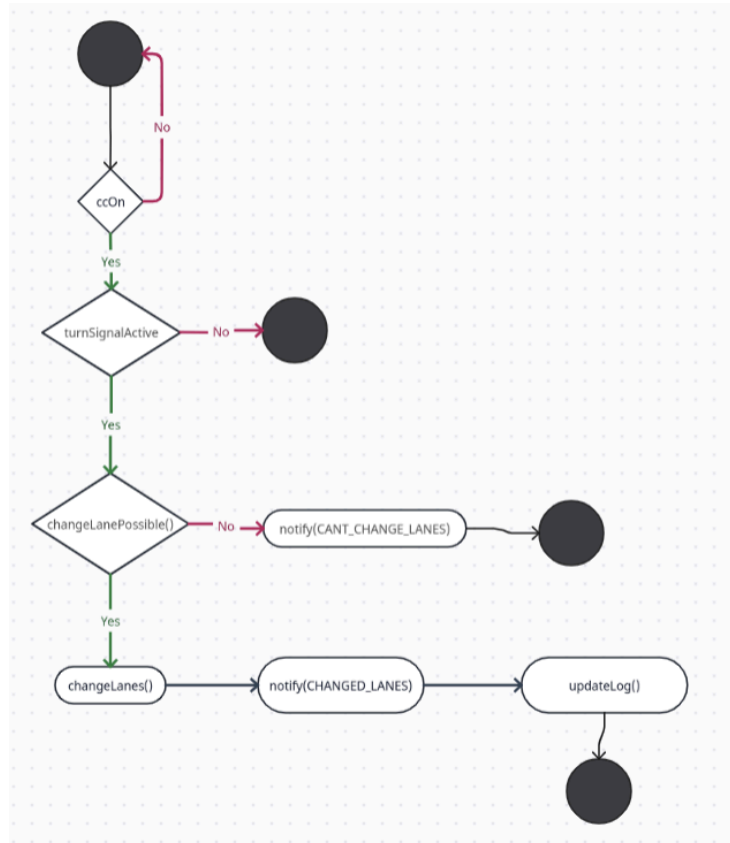
1. The user signs out. The display returns to normal state.
2. The vehicle is powered off. The display returns to normal state.

4.2 Activity Diagrams

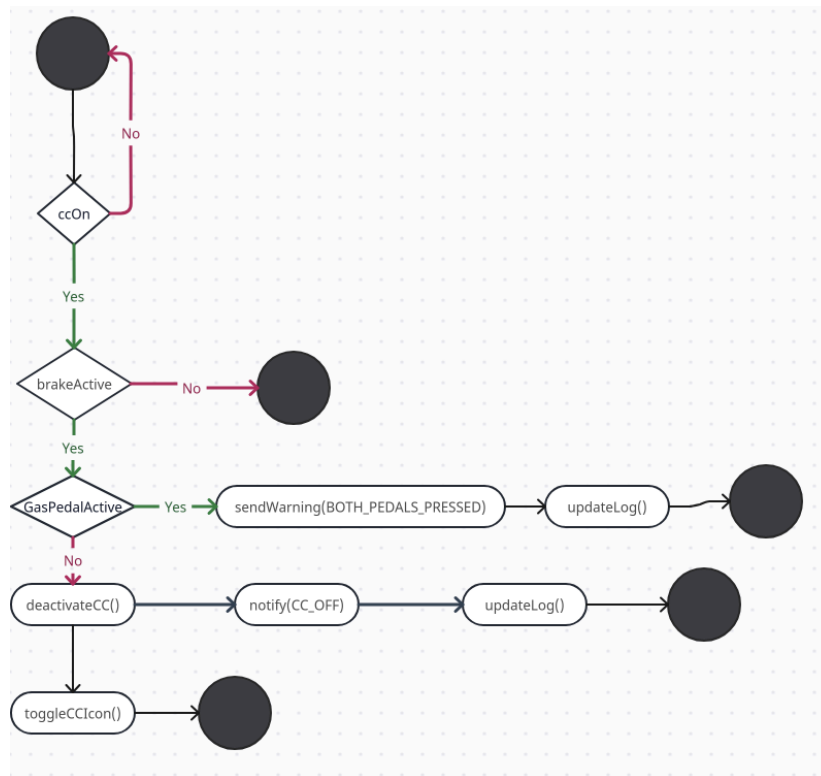
4.2.1 Sensors Detect an Object in the Path of the Vehicle



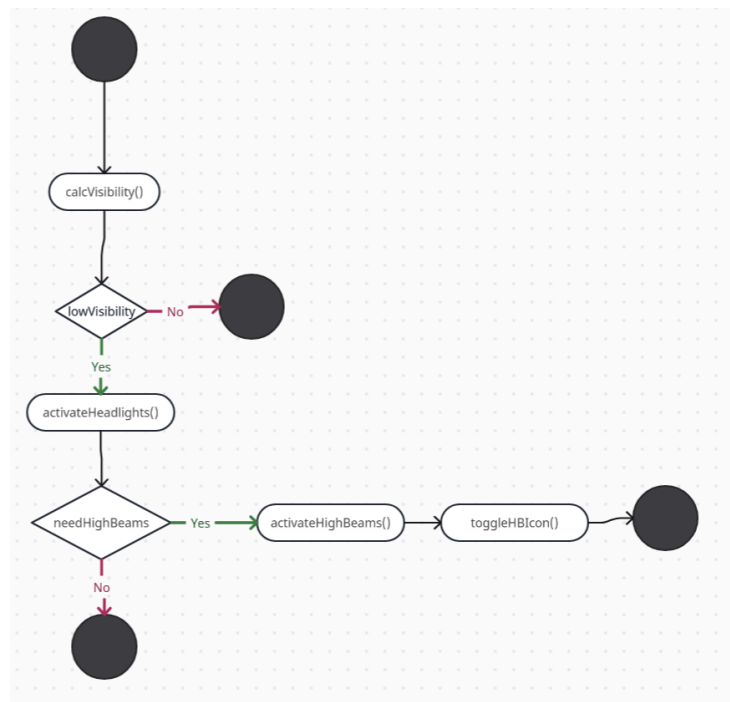
4.2.2 Driver Activates Automatic Lane Change While in Cruise Control



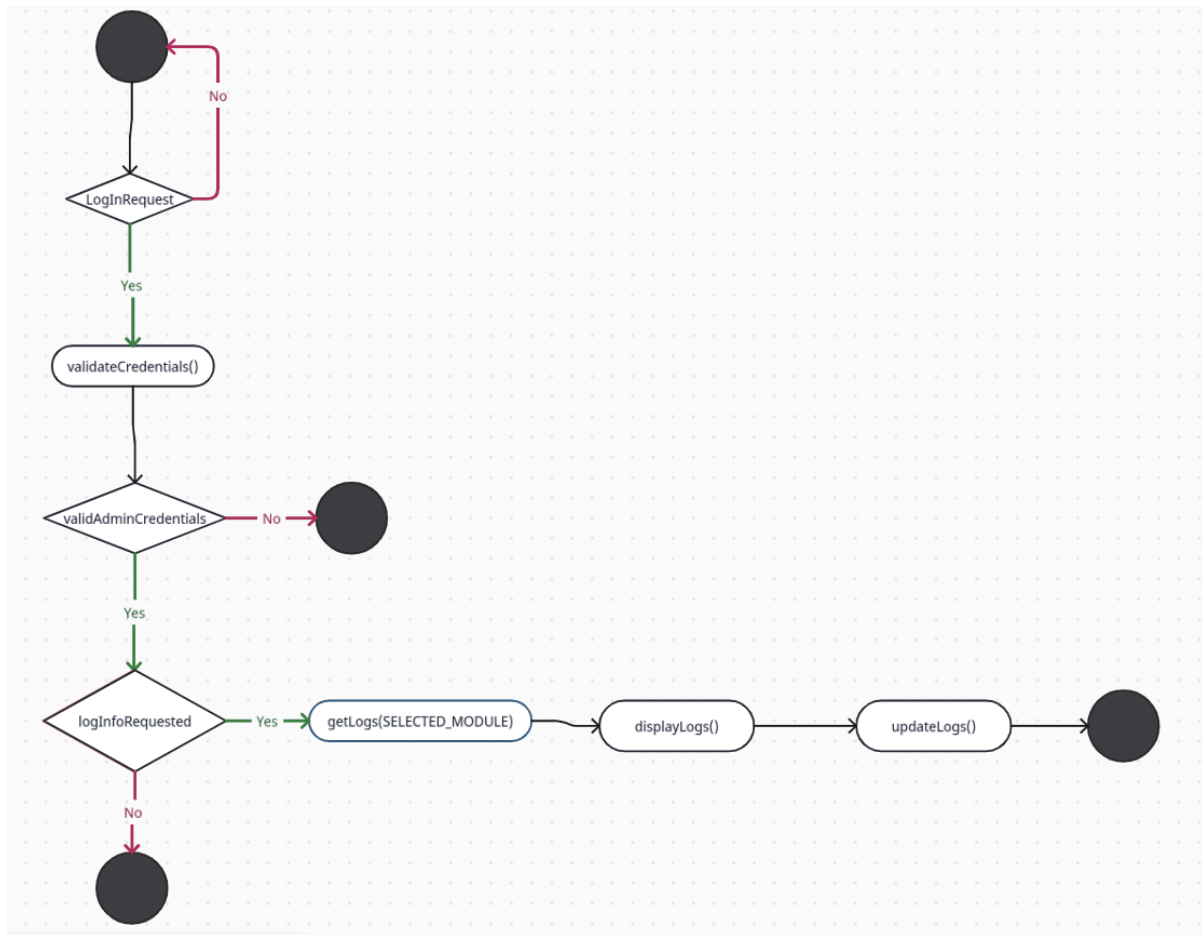
4.2.3 Driver Engages Brakes While in Cruise Control



4.2.4 Light Sensor Activates Headlights

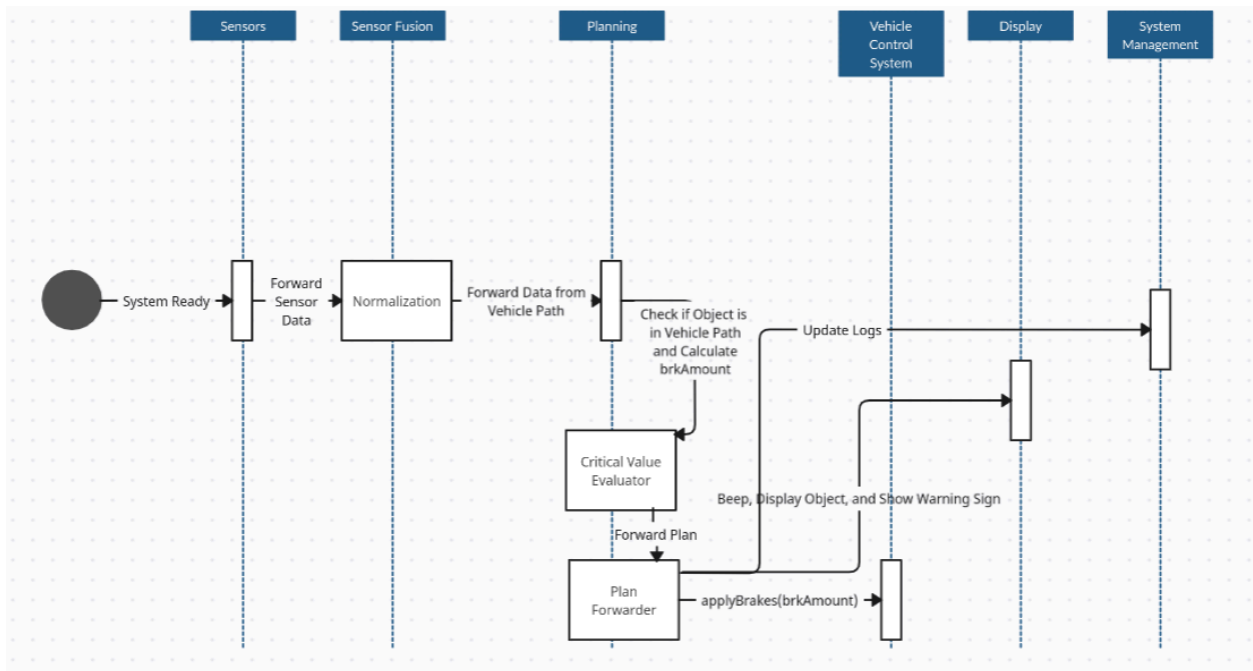


4.2.5 Technician Requests Log Files

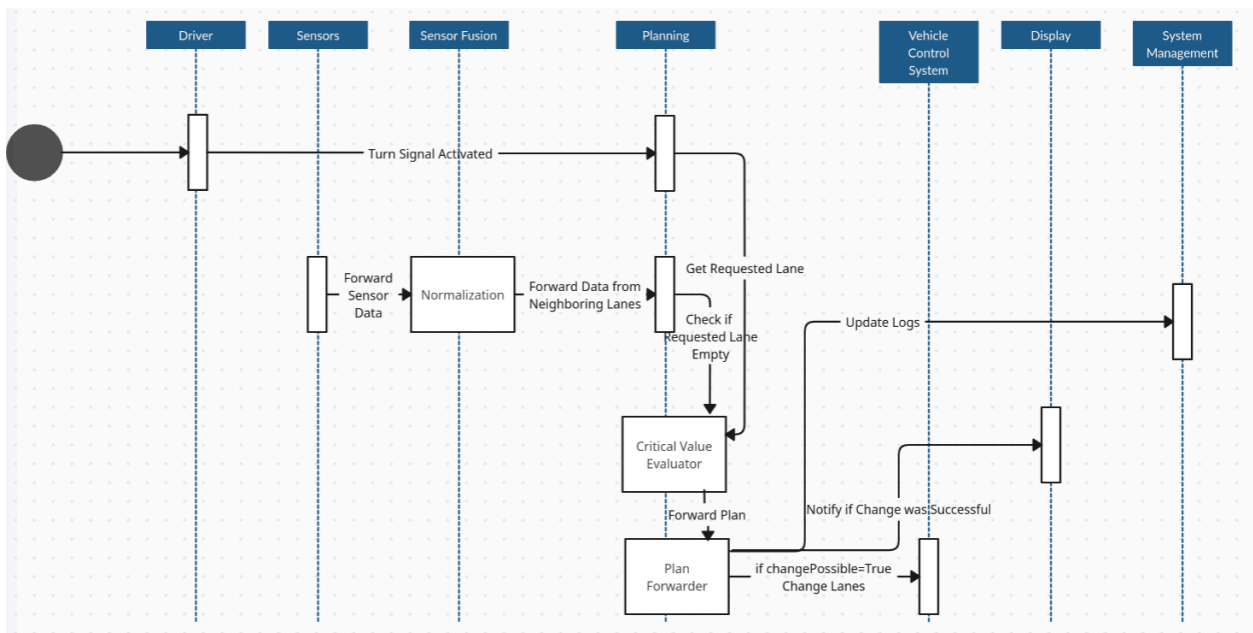


4.3 Sequence Diagrams

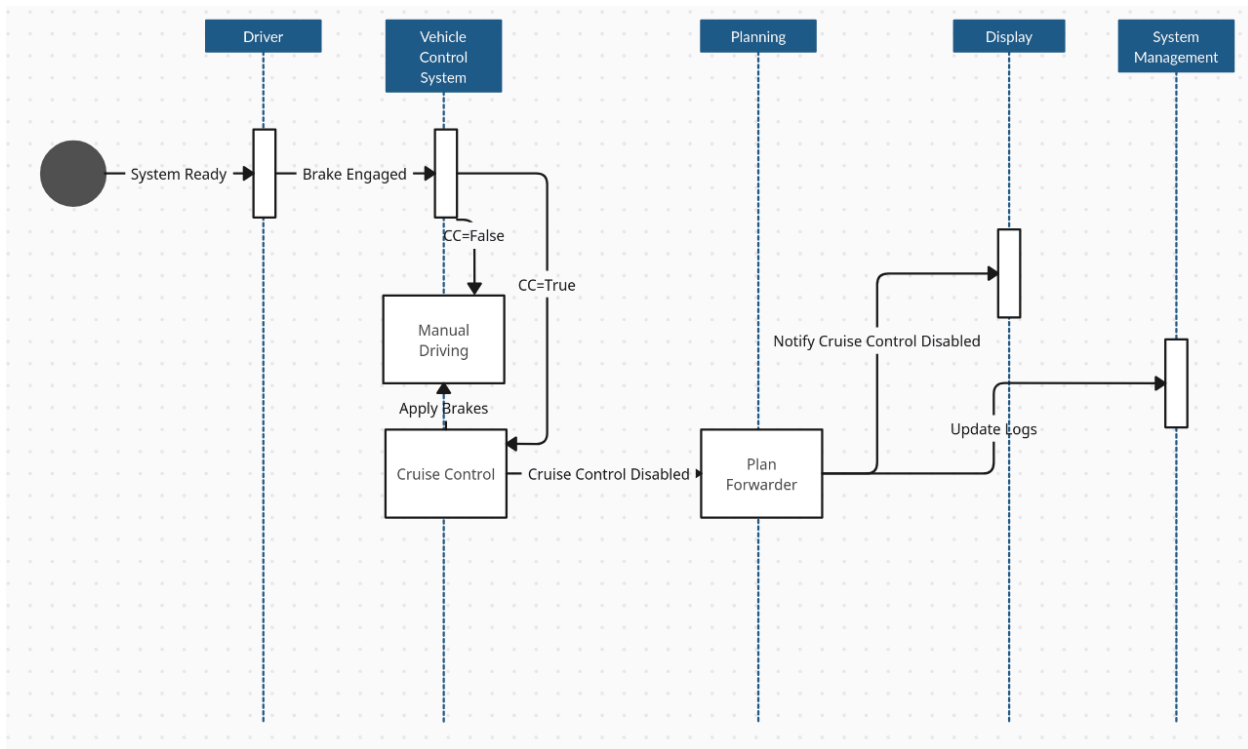
4.3.1 Sensors Detect an Object in the Path of the Vehicle



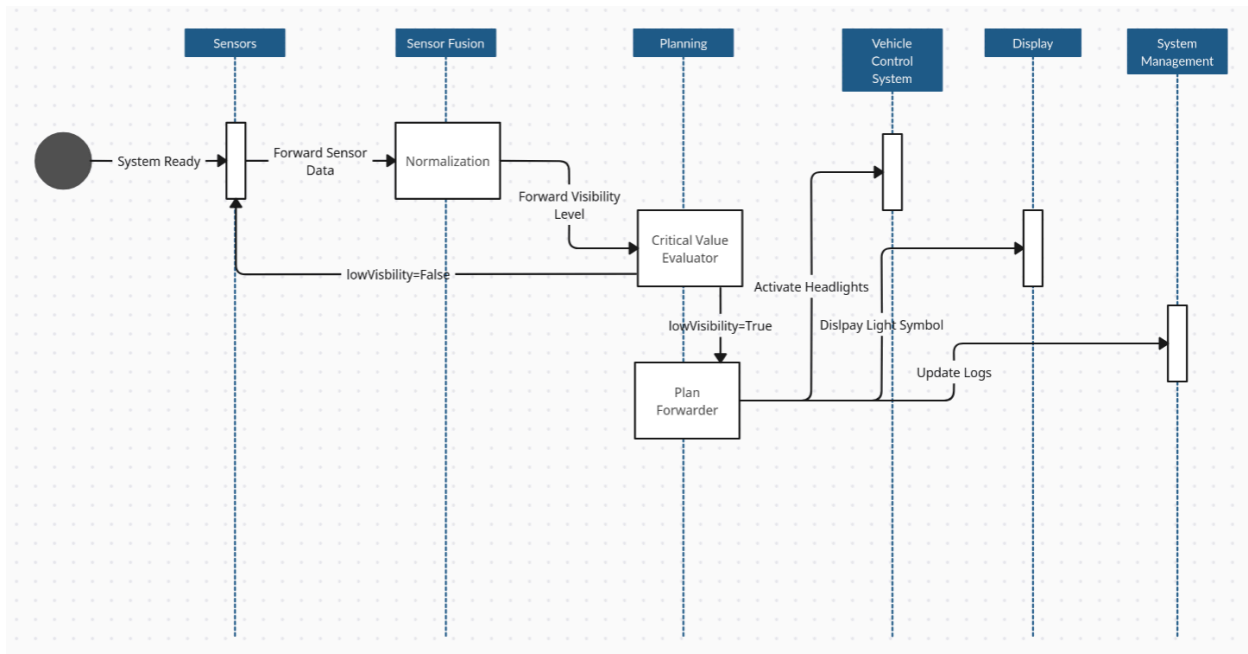
4.3.2 Driver Activates Automatic Lane Change While in Cruise Control



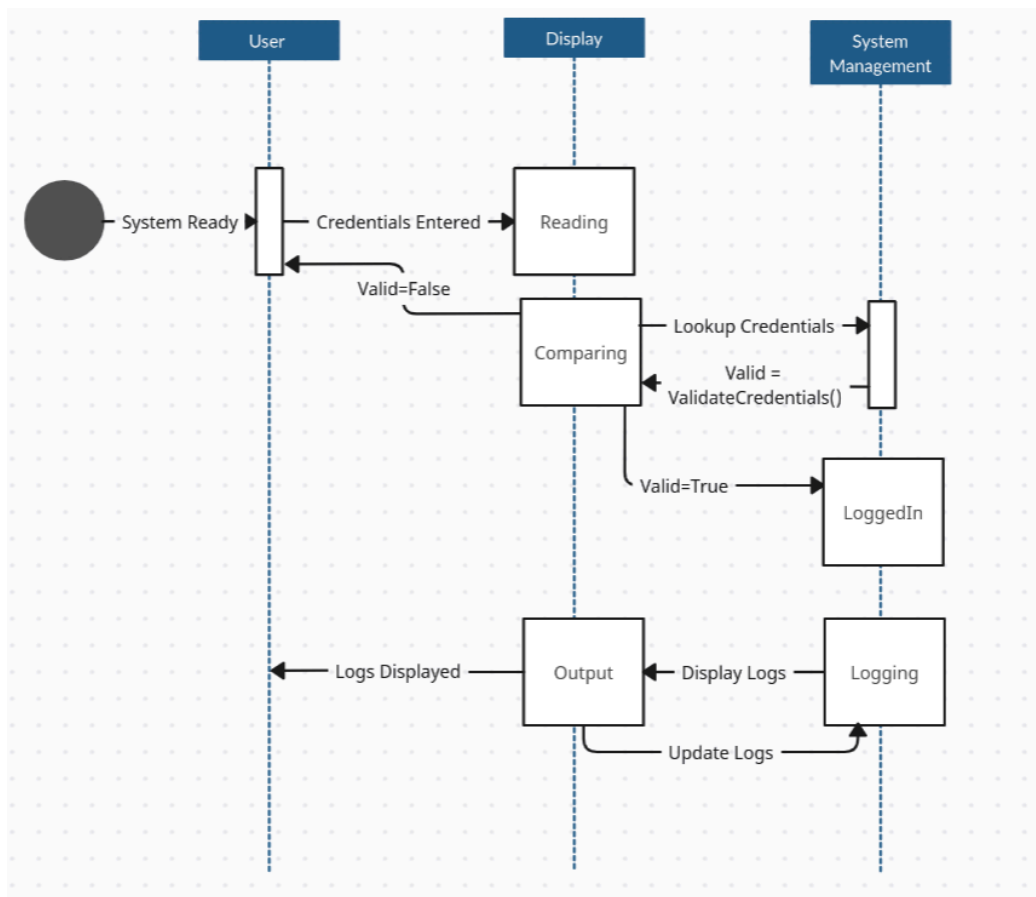
4.3.3 Driver Engages Brakes While in Cruise Control



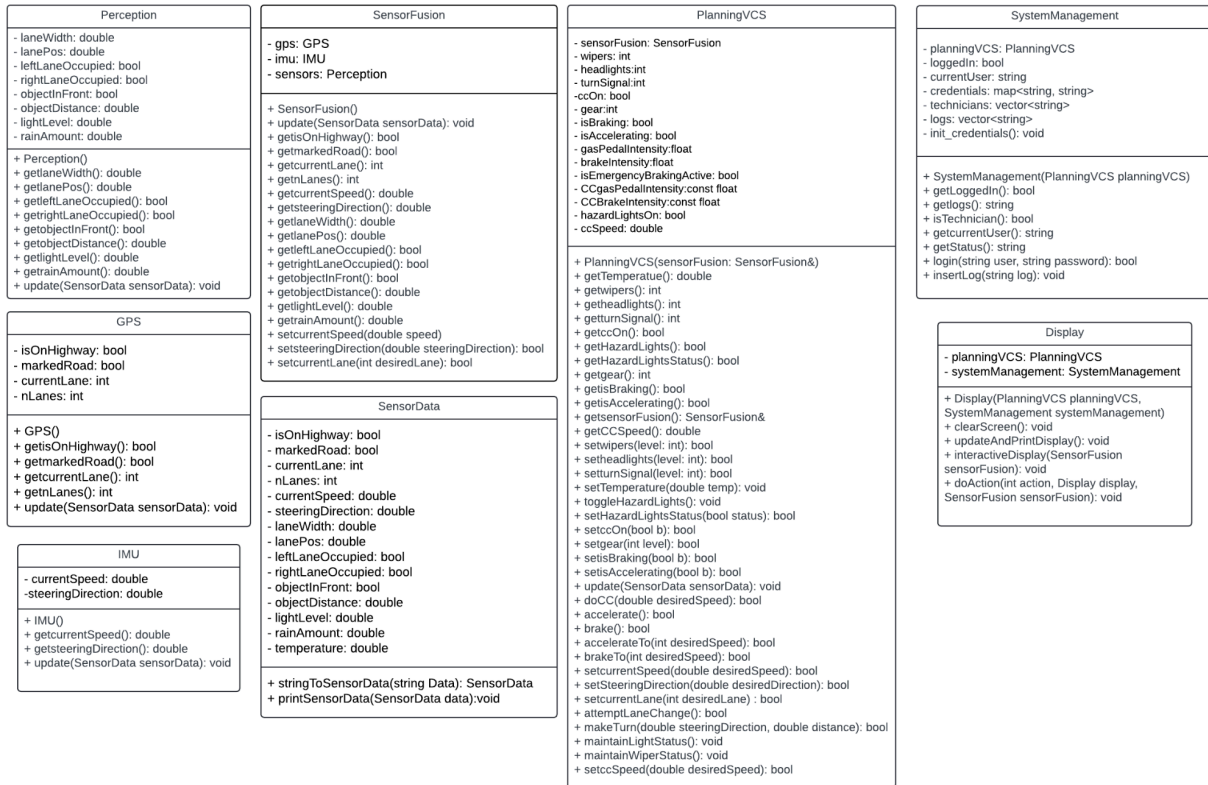
4.3.4 Light Sensor Activates Headlights



4.3.5 Technician Requests Log Files

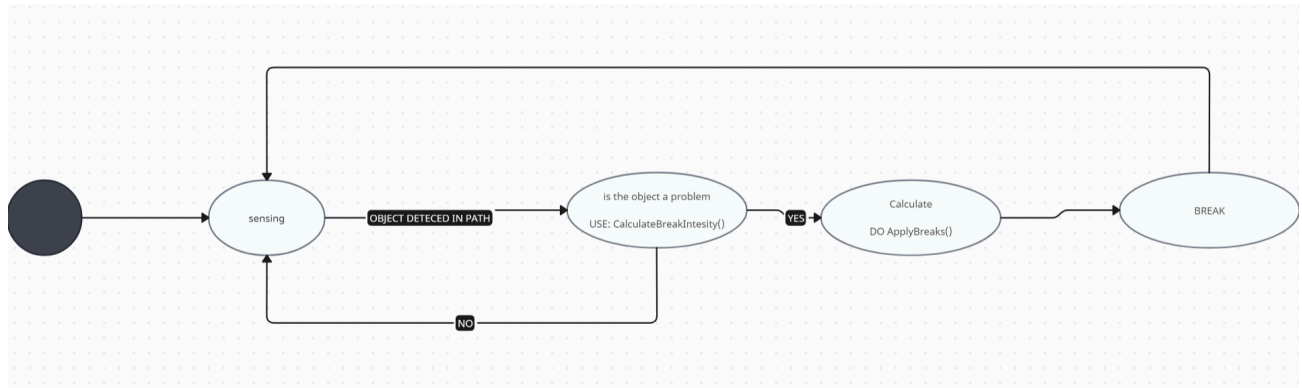


4.4 Classes

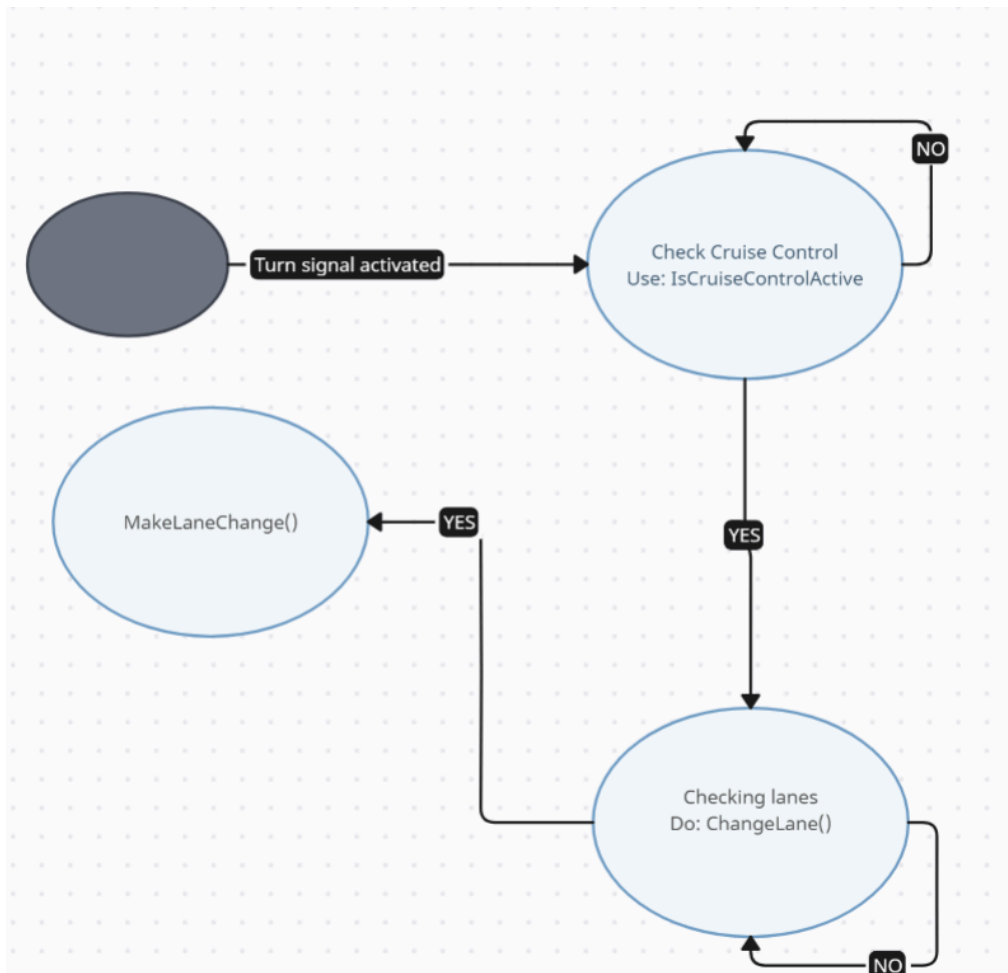


4.5 State Diagrams

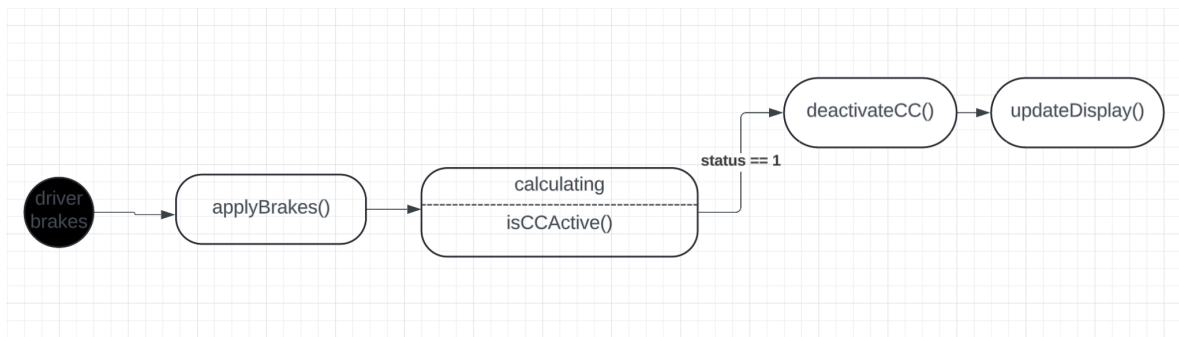
4.5.1 Sensors detect object(s) in path of vehicle



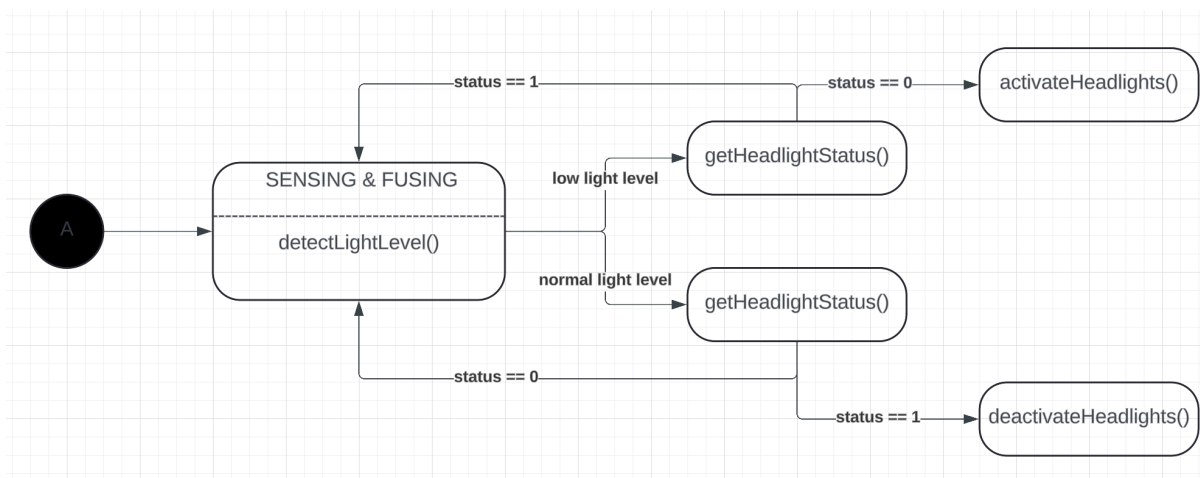
4.5.2 Driver requests automatic lane change



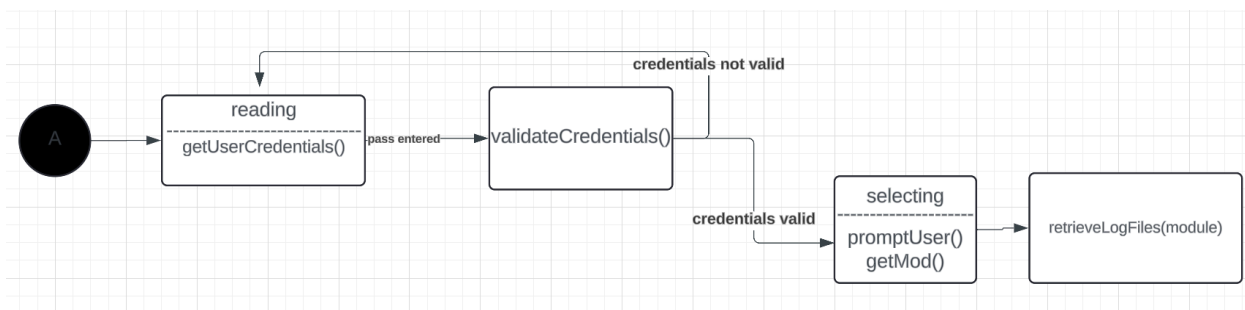
4.5.3 Driver Engages Brakes While in Cruise Control



4.5.4 Light Sensors Activate Headlights



4.5.5 Technician Requests Log Files

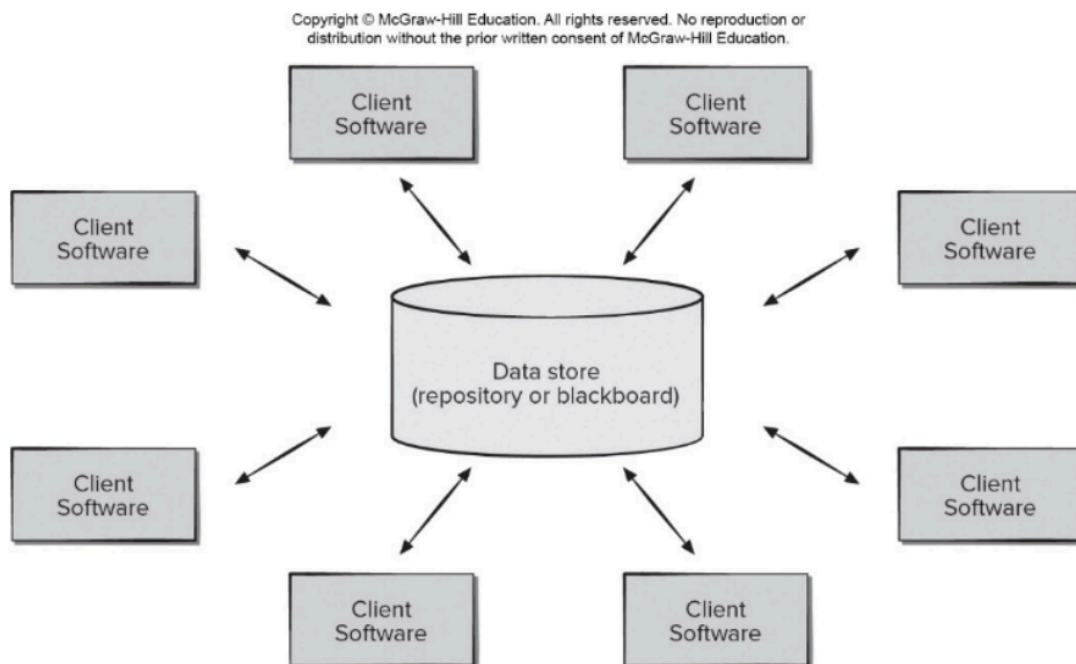


5. Design

5.1 Software Architecture

The system's software architecture will consist of several objects representing the modules of the system. Details on the class interface for each object can be found in [section 5.2](#). Due to the strong connection between the sensor fusion and planning modules and planning and vehicle control system modules, the planning module will be divided among the two. Hence, the system will be represented by the objects of the following class interfaces: the Driver interface, Technician interface, Sensor Fusion-Planning interface, and VCS-Planning interface. The objects instantiated from these classes will not be organized hierarchically, but data will typically flow iteratively. Any data or functionality that must be accessed by other objects will be declared as public, while everything used exclusively by the object itself will be declared as private, which will provide a simpler interface for other objects to use.

5.1.1 Data Centered Architecture



The data centered architecture uses a central, shared data store that is utilized by various software components for communication.

Pros:

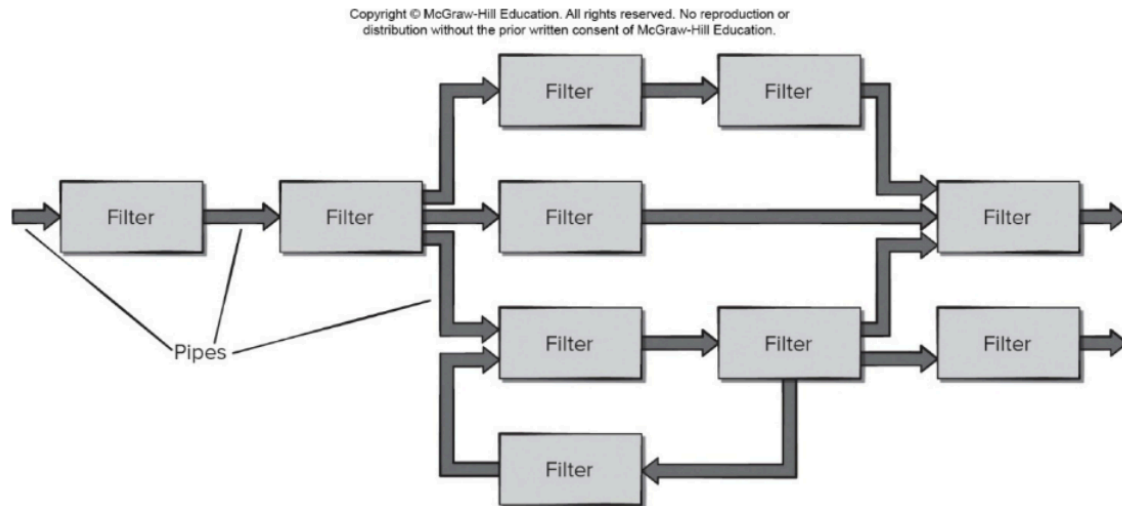
- Only have to change data in one, centralized place accessible to all components.
- Rules for modifying data can be put in place.

Cons:

- Poses synchronization difficulties with concurrent accesses to data store.
- The data store is a single point of failure for the architecture.

Feasibility: While a data centered architecture would be possible to implement for this system, most data is module-specific and giving all components access to all variables would introduce unnecessary complexity to the architecture.

5.1.2 Data Flow Architecture



The data flow architecture connects various components using pipes, which transforms input data and feeds it to other components.

Pros:

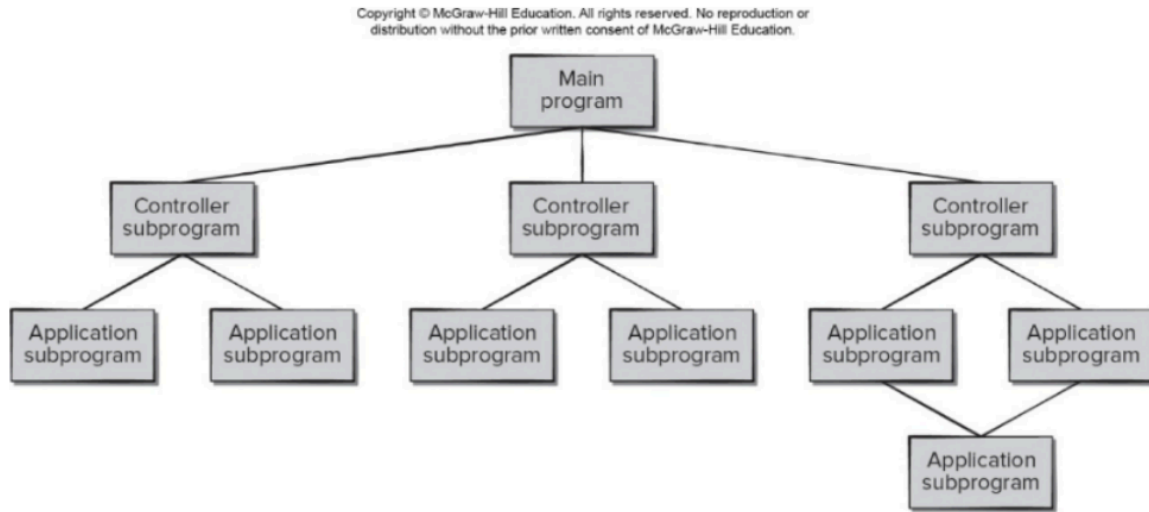
- Easy to design and implement.
- Can be divided into independently functioning subsystems.

Cons:

- Rigid, hard to modify.
- Critical points of failure can bring down system.
- High latency and low throughput.

Feasibility: While the potential modularity and ease of design of the data flow architecture makes it an attractive option, its high latency makes it an infeasible architecture for the system. The system requires low latency in order to react to new data at fast speeds, and the data flow architecture cannot provide such a thing.

5.1.3 Call Return Architecture



The call return architecture consists of a main program that invokes controller subprograms, which invoke application subprograms.

Pros:

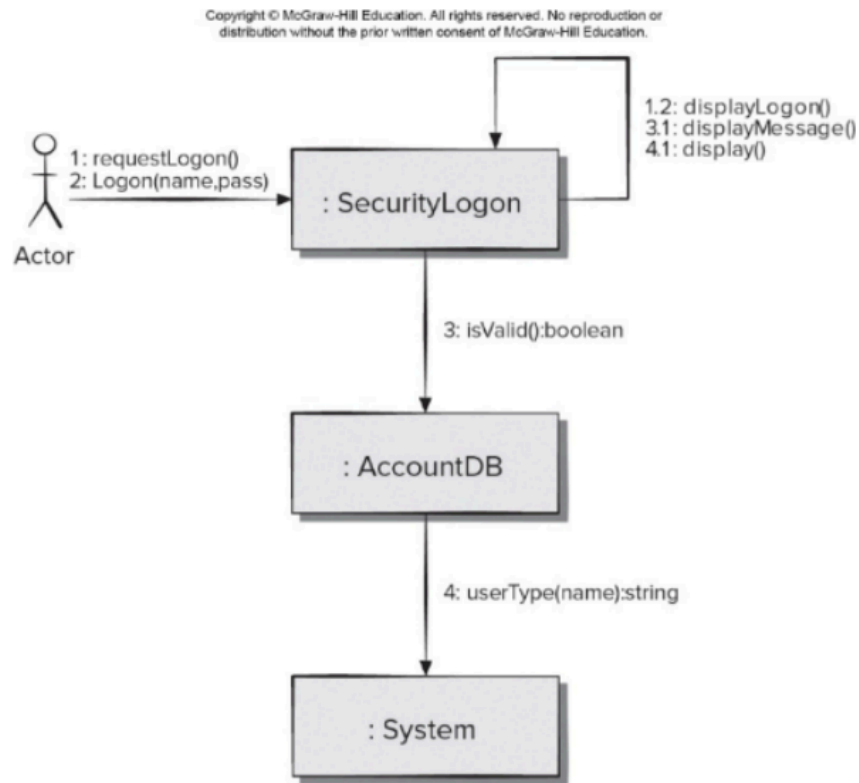
- Easy to scale existing capabilities.
- Code is modular and runs independently, making modification easy.

Cons:

- Communication between modules is more difficult.
- Dependencies between modules may be difficult to implement.

Feasibility: The call return architecture is certainly not feasible for the system, as the system depends heavily on intermodule communication, which proves to be fairly difficult to implement using this architecture.

5.1.4 Object-Oriented Architecture



The object-oriented architecture provides an abstraction for each component or module which encapsulates all data and operations related to it.

Pros:

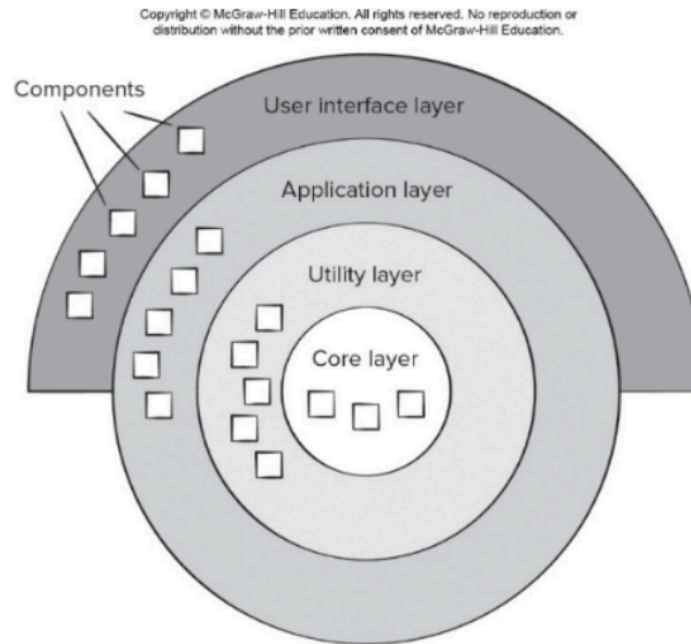
- Objects can be easily modified.
- Objects are easier to work with than separated code and data (next layer of abstraction).
- Private and public fields allow for selective access and modification of code and data.

Cons:

- Difficult to develop sophisticated objects.

Feasibility: The object-oriented architecture would be feasible for the system given its clear separation of components and combination of data and functionality. The system is already divided into modules with clear functions and a state that needs to be maintained, and objects would allow these to be encapsulated into one abstracted unit per module.

5.1.5 Layered Architecture



The layered architecture splits components into different “layers” that each perform a different range of functions independently of one another.

Pros:

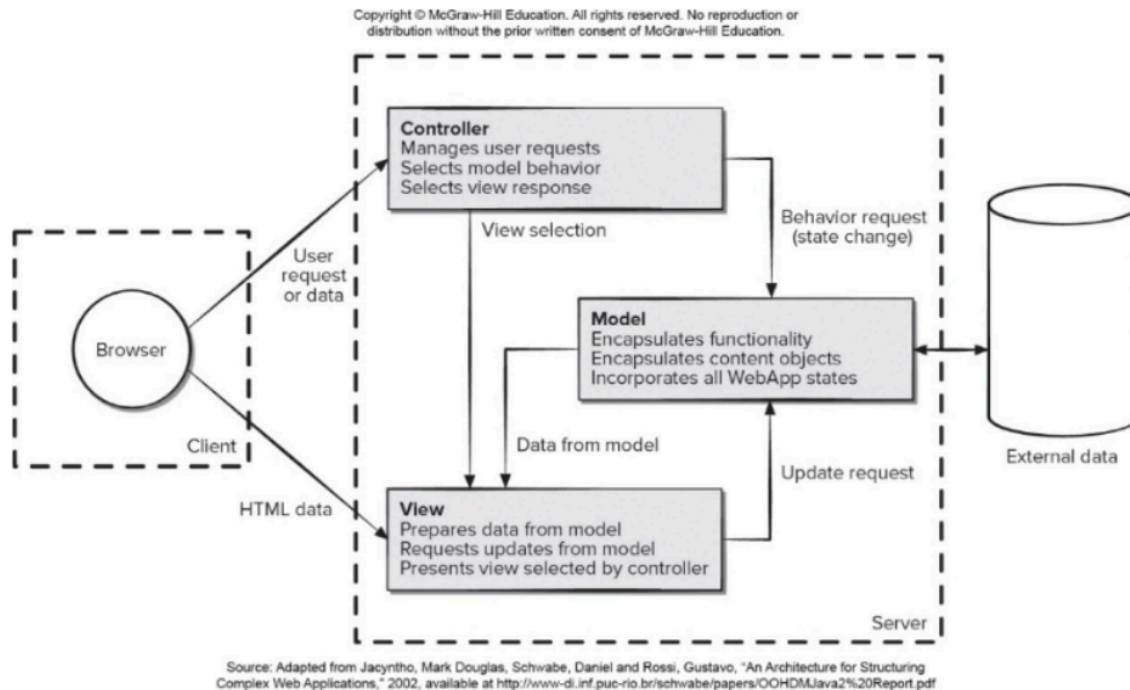
- Layers are independent from one another.
- Each layer can be modified without breaking the architecture.
- Separate layers make testing easy.

Cons:

- Difficult to scale.
- Additional complexity comes at the cost of speed.
- Sequentiality of architecture does not allow for more complex interactions between layers.

Feasibility: The layered architecture would not be a feasible architecture for the system because of its lack of support for complex interactions between layers. Typically, each layer in a layered architecture has one or no input layers and one or no output layers. Since each layer in the system may need several output layers in some cases, this architecture would not be able to aptly represent the functionality of the system.

5.1.6 Model View Controller Architecture



The model view controller architecture into three logical components: the model, the view, and the controller. The model serves as the “brain” of the system and handles all logic and computations. The view handles the presentation of data and provides an interface for the user. The controller facilitates communication between the model and the view and guides inbound and outbound data traffic.

Pros:

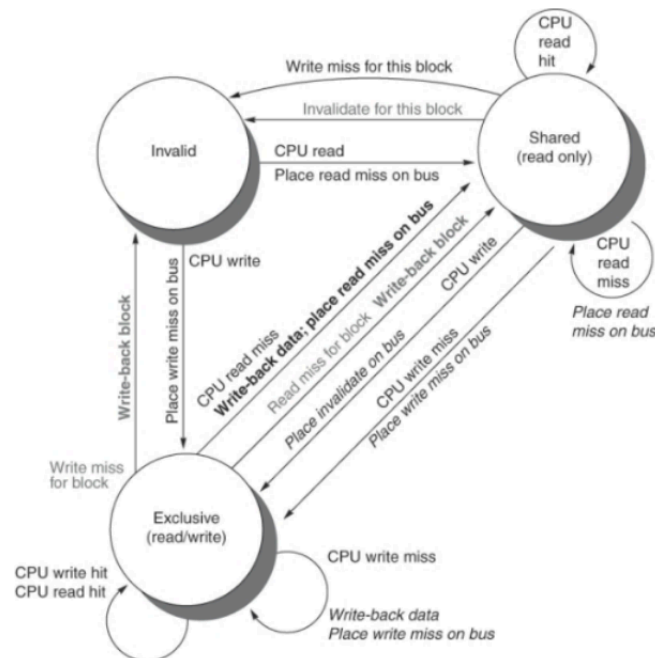
- Code is easy to maintain and update (split into front and back ends).
- Components can be tested and modified independently.

Cons:

- Increased complexity due to additional layers of abstraction.
- Slow for data to reach the view component.

Feasibility: The model view controller architecture would not be a feasible architecture for the system because the abstractions it provides does not adequately divide the system. Using the model view controller architecture, most of the system would be encapsulated by the model and the view/controller would be fairly simple in comparison. Thus, the model view controller abstraction is a poor fit for the system and would add unnecessary complexity.

5.1.7 Finite State Machine Architecture



The finite state machine architecture represents the system as a directed graph composing a set of states with various transitions between them depending on values from the previous state or previous input.

Pros:

- Flexible and clear definition of the relationship between different components.
- Easy to determine if a state is reachable and how it can be reached.

Cons:

- Can grow in complexity very fast.
- Can be hard to separate components into states.

Feasibility: A finite state machine architecture would not be a feasible architecture for the system. The system transitions between many states and can perform different actions based on what the previous states were. Since in a finite state machine there is no concept of specific sequences, there is not a clear division of when functions should be executed based on the current action being performed.

5.1.8 Chosen Architecture

Given the capabilities of each of the above architectures, the system will use an object-oriented architecture. The system has already been designed as a collection of modules each with their own data and functionality. Thus, these modules can very easily be translated into classes. Data and functionality that must be utilized by outside modules can be declared as public, while those used only inside the module can be declared as private. Each module will have an interface in which the functionality of each is clear and how they interact and use one another will be clear.

5.2 Interface Design

5.2.1 Data Aggregation Interface

The Data Aggregation interface will act as the primary data source for the other components. It combines the localization (IMU & GPS classes), perception, and sensor fusion modules into one simple interface.

The Data Aggregation interface will be invoked exclusively by the Planning-VCS interface.

Classes: IMU, GPS, Perception, SensorFusion

Data:

- Bool: isOnHighway, markedRoad, leftLaneOccupied, rightLaneOccupied, objectInFront
- Int: currentLane, nLanes
- Double: currentSpeed, laneWidth, lanePos, objectDistance, lightLevel, rainAmount, steeringDirection

Main Functions: update, getters for data

5.2.2 Planning-VCS Interface

The Planning-VCS interface will be responsible for the vehicle's reactions to user input and collected data. It will communicate directly with the data aggregation interface in order to respond in real time to events and can be invoked manually via the interaction interface.

Classes: PlanningVCS

Data:

- Bool: ccOn, isBraking, isAccelerating
- Int: wipers, headlights, turnSignal, gear

Main Functions: setCurrentSpeed, setsteeringDirection, makeTurn, doCC, attemptLaneChange

5.2.3 Interaction Interface

The Interaction interface will be responsible for all user interaction with the system. This encompasses both the system management and display modules. The SystemManagement class will oversee the configuration, authorization, maintenance, and monitoring of the system.

Classes: SystemManagement, Display

Data:

- Bool: loggedIn
- String: currentUser
- Vector<string>: technicians, logs
- Map<string, string>: credentials

Main Functions:

- **TECHNICIAN:** getLogs, getStatus
- **DRIVER:** read data, login, perform actions

5.3 Component-Level Design

5.3.1 Data Aggregation Interface

```
1  /* dataAggregation.hpp
2     This file contains the class definitions for the GPS, IMU, Perception, and SensorFusion classes.
3     These classes are used to aggregate data from various sensors and provide a unified interface for accessing the data.
4  */
5
6  #ifndef DATAAGGREGATION_HPP
7  #define DATAAGGREGATION_HPP
8
9  #include <string>
10
11 using namespace std;
12
13 struct SensorData {
14     bool isOnHighway;
15     bool markedRoad;
16     int currentLane;
17     int nLanes;
18     double currentSpeed;
19     double steeringDirection;
20     double laneWidth;
21     double lanePos;
22     bool leftLaneOccupied;
23     bool rightLaneOccupied;
24     bool objectInFront;
25     double objectDistance;
26     double lightLevel;
27     double rainAmount;
28     double temperature;
29 };
30
31 SensorData stringToSensorData(string data);
32 void printSensorData(SensorData data);
33
34 class GPS {
35     friend class SensorFusion;
36     private:
37     bool isOnHighway; // True if the vehicle is on a highway, false otherwise
38
39     bool markedRoad; // True if the road is marked, false otherwise
40
41     int currentLane; // The lane the vehicle is currently in, numbered from right to left, 0 to nLanes - 1
42
43     int nLanes; // The number of lanes on the vehicle's side of the road
44
45     public:
46     GPS(); // Constructor for GPS
47
48     bool getisOnHighway() const; // Return isOnHighway
49
50     .. ..
```

```

51     bool getmarkedRoad() const; // Return markedRoad
52
53     int getcurrentLane() const; // Return currentLane
54
55     int getnLanes() const; // Return nLanes
56
57     void update(SensorData sensorData); // Accesses the location of the vehicle and updates the private state of GPS
58
59 };
60
61 class IMU {
62     friend class SensorFusion;
63 private:
64     double currentSpeed; // The current speed of the car in MPH
65
66     double steeringDirection; // The direction that the car is being steered. Ranges from -1 (left) to 1 (right). 0 is straight.
67
68 public:
69     IMU(); // Constructor for IMU
70
71     double getcurrentSpeed() const; // Return currentSpeed;
72
73     double getsteeringDirection() const; // Return steeringDirection
74
75     void update(SensorData sensorData); // Updates the private state of IMU with current vehicle data
76
77 };
78
79 class Perception {
80     friend class SensorFusion;
81 private:
82     double laneWidth; // Width of the lane in feet
83
84     /* Position of the vehicle in the lane. A value of 0 means that
85        the left side of the car is on the left line of the lane while
86        a value of laneWidth-CAR_WIDTH means that the right side of the
87        car is on the right line of the lane.
88     */
89     double lanePos;
90
91     /* True if there is an object in the lane left of the vehicle (if
92        there is a lane left of the vehicle). This includes any objects
93        in the vehicle's blindspot on the left side.
94     */
95     bool leftLaneOccupied;
96
97     /* True if there is an object in the lane right of the vehicle (if
98        there is a lane right of the vehicle). This includes any objects
99        in the vehicle's blindspot on the right side.
100    */

```

```

101     bool rightLaneOccupied;
102
103     bool objectInFront; // True if there is an object detected in front of the vehicle.
104
105     double objectDistance; // The distance between the front of the vehicle and the object in front of it (if there is one).
106
107     double lightLevel; // The light level detected by the vehicle's sensors
108
109     double rainAmount; // The amount of rain detected by the vehicle's sensors
110
111     public:
112     Perception(); // Constructor for Perception
113
114     double getLaneWidth() const; // Return laneWidth
115
116     double getLanePos() const; // Return lanePos
117
118     bool getLeftLaneOccupied() const; // Return leftLaneOccupied
119
120     bool getRightLaneOccupied() const; // Return rightLaneOccupied
121
122     bool getObjectInFront() const; // Return objectInFront
123
124     double getObjectDistance() const; // Return objectDistance
125
126     double getLightLevel() const; // Return lightLevel
127
128     double getRainAmount() const; // Return rainAmount
129
130     void update(SensorData sensorData); // Updates the private state of Perception with current vehicle data
131
132 };
133
134 class SensorFusion {
135     private:
136
137     GPS gps;
138     IMU imu;
139     Perception sensors;
140
141     public:
142     SensorFusion(); // Constructor for SensorFusion
143
144     void update(SensorData sensorData); // Updates the private state of SensorFusion with current vehicle data
145
146     bool getIsOnHighway() const; // Return isOnHighway
147
148     bool getMarkedRoad() const; // Return markedRoad
149
150     int getCurrentLane() const; // Return currentLane

```

```
151
152     int getnLanes() const; // Return nLanes
153
154     double getcurrentSpeed() const; // Return currentSpeed;
155
156     double getsteeringDirection() const; // Return steeringDirection
157
158     double getlaneWidth() const; // Return laneWidth
159
160     double getlanePos() const; // Return lanePos
161
162     bool getleftLaneOccupied() const; // Return leftLaneOccupied
163
164     bool getrightLaneOccupied() const; // Return rightLaneOccupied
165
166     bool getobjectInFront() const; // Return objectInFront
167
168     double getobjectDistance() const; // Return objectDistance
169
170     double getlightLevel() const; // Return lightLevel
171
172     double getrainAmount() const; // Return rainAmount
173
174     bool setcurrentSpeed(double speed); // Set currentSpeed
175
176     bool setsteeringDirection(double direction); // Set currentSpeed
177
178     bool setcurrentLane(int desiredLane); // Set currentLane
179
180 };
181
182 #endif // DATAAGGREGATION_HPP
```

5.3.2 Planning-VCS Interface

```
1  /*planningvcs.cpp*/
2  #include <iostream>
3  #include "planningVCS.hpp"
4  #include "dataAggregation.hpp"
5
6  #include "interaction.hpp"
7  PlanningVCS::PlanningVCS(SensorFusion &sensorFusion) //set default values
8      : sensorFusion(sensorFusion) {
9      wipers = -1; //wipers off
10     headLights = 0; //headlights off
11     temperature = 72; //default temperature is 72 degrees
12     turnSignal = -1; //turn signal neutral
13     ccOn = false; //Cruise Control not on
14     gear = 0; //gear in park
15     isBraking = false; //not braking
16     isAccelerating = false; //not accelerating
17     ccSpeed = 60; // Default speed for cruise control is 60 mph
18     hazardLightsOn = false; // the lights are not on
19 }
20
21
22 int PlanningVCS::getwipers() const { //get status of wipers, -1 is off, 1 is on, 2 is on and in auto mode
23     return wipers;
24 }
25
26 double PlanningVCS::getTemperature() const {
27     return temperature;
28 }
29
30 int PlanningVCS::getheadlights() const { //get headlight status, 0 if off, 1 if low beams on, 2 if high beams on
31     return headLights;
32 }
33
34 int PlanningVCS::getturnSignal() const { //get turn signal status. -1 off, 0 left, 1 right
35     return turnSignal;
36 }
37
38 bool PlanningVCS::getccOn() const { //get Cruise Control status. true is on
39     return ccOn;
40 }
41
42 int PlanningVCS::getgear() const { //return gear int. 0 park, 1 drive, 2 neutral, 3 drive
43     return gear;
44 }
45
46 bool PlanningVCS::getisBraking() const { //Returns true if vehicle is braking
47     return isBraking;
48 }
49
50
```

```

51 bool PlanningVCS::getisAccelerating() const { //Returns true if vehicle is accelerating
52     return isAccelerating;
53 }
54 bool PlanningVCS::getHazardLightsStatus() const { //return true if lights are on
55     return hazardLightsOn;
56 }
57
58 SensorFusion& PlanningVCS::getsensorFusion() const { //Returns sensor data
59     return sensorFusion;
60 }
61
62 double PlanningVCS::getccSpeed() const { // Returns ccSpeed
63     return ccSpeed;
64 }
65
66 bool PlanningVCS::setwipers(int level) { //set wipers in range form -1 to 1
67     if (level >= -1 && level <= 1) {
68         wipers = level;
69         return true;
70     }
71     return false;
72 }
73
74
75 bool PlanningVCS::setheadlights(int level) { //set headlights in range from 0 to 2
76     if (level >= 0 && level <= 2) {
77         headlights = level;
78         return true;
79     }
80     return false;
81 }
82
83 bool PlanningVCS::setcurrentLane(int lane) { //set current lane in range from 0 to nLanes - 1
84     if (lane >= 0 && lane < sensorFusion.getnLanes()) {
85         sensorFusion.setcurrentLane(lane);
86         return true;
87     }
88     return false;
89 }
90
91 bool PlanningVCS::setturnSignal(int level) { //set turn signal in range from -1 to 1
92     if (level >= -1 && level <= 1) {
93         turnSignal = level;
94         return true;
95     }
96     return false;
97 }
98
99 bool PlanningVCS::setccOn(bool b) { //set the mode for cruise control, on or off
100     ccOn = b;

```

```

101     return true;
102 }
103
104 bool PlanningVCS::setccSpeed(double speed) { // Sets the ccSpeed between 50 and 70
105     if (speed < 50 || speed > 70) return false;
106     ccSpeed = speed;
107     return true;
108 }
109
110 bool PlanningVCS::setgear(int level) { //set the gear level in range from 0 to 3
111     if (level >= 0 && level <= 3) {
112         gear = level;
113         return true;
114     }
115     return false;
116 }
117
118 bool PlanningVCS::setisBraking(bool b) { //set isBraking to true or false
119     isBraking = b;
120     return true;
121 }
122
123 bool PlanningVCS::setisAccelerating(bool b) { //set isAccelerating to true or false
124     isAccelerating = b;
125     return true;
126 }
127
128 bool PlanningVCS::setcurrentSpeed(double speed) {
129     if (speed > 110) {
130         cout << "Error: Speed limit exceeded. Setting speed to 70 mph." << endl;
131         sensorFusion.setcurrentSpeed(70);
132         return true; // Indicating that the speed has been set, albeit not to the requested speed
133     }
134     return sensorFusion.setcurrentSpeed(speed);
135 }
136
137 bool PlanningVCS::setsteeringDirection(double direction) {
138     return sensorFusion.setsteeringDirection(direction);
139 }
140 void PlanningVCS::toggleHazardLights() {
141     hazardLightsOn = !hazardLightsOn;
142 }
143
144 void PlanningVCS::setTemperature(double temp) {
145     if (temp >= 50 && temp <= 80) {
146         temperature = temp;
147     } else {
148         cout << "Invalid temperature. Please choose a value between 50-80 degrees." << endl;
149     }
150 }

```



```

151 void PlanningVCS::update(SensorData sensorData) {
152     sensorFusion.update(sensorData);
153     // Check if the vehicle is on a highway
154     if (sensorData.currentSpeed != 0) {
155         setgear(1);
156     } else {
157         setgear(0);
158     }
159     if (!sensorFusion.getisOnHighway()) {
160         // If the vehicle is not on a highway, ensure cruise control is turned off
161         setccOn(false);
162     } else {
163         // If the vehicle is on a highway, proceed with cruise control logic
164         if (ccOn && getturnSignal() == -1) {
165             // If cruise control is on and no turn signal is active, manage speed
166             double currentSpeed = sensorFusion.getCurrentSpeed();
167             double desiredSpeed = getccSpeed(); // Assuming there is a method to fetch the desired speed
168
169             if (currentSpeed < desiredSpeed) {
170                 accelerateTo(desiredSpeed);
171             } else if (currentSpeed > desiredSpeed) {
172                 brakeTo(desiredSpeed);
173             }
174         } else if (ccOn && (getturnSignal() == 0 || getturnSignal() == 1)) {
175             // If the turn signal is on, attempt to change lanes
176             attemptLaneChange();
177         }
178     }
179 }
180 if (sensorFusion.getLightLevel() <= HIGHBEAM_THRESHOLD)
181     setheadlights(2);
182 else if (sensorFusion.getLightLevel() <= LOWBEAM_THRESHOLD)
183     setheadlights(1);
184 else
185     setheadlights(0);
186 if (sensorFusion.getrainAmount() >= RAIN_THRESHOLD)
187     setwipers(1);
188 else
189     setwipers(-1);
190 temperature = sensorData.temperature;
191 }
192
193
194 bool PlanningVCS::doCC(double desiredCCSpeed) { //Initiate cruise control
195     if(desiredCCSpeed > 90 || desiredCCSpeed < 25 || !sensorFusion.getisOnHighway()||!getturnSignal()) return false; //check if speed is within sa
196     while(ccOn && sensorFusion.getmarkedRoad()){ //double check cc is on and the current road is marked
197         //decide whether to accelerate/decelrate the vehicle to desired speed
198         if(sensorFusion.getCurrentSpeed() < desiredCCSpeed)
199             {accelerateTo(desiredCCSpeed); }
200         else {

```

```

201         brakeTo(desiredCCSpeed);
202     }
203     //if turn signal is activated, attempt to change lanes
204     if(getTurnSignal() == 0 || getTurnSignal() == 1){
205         attemptLaneChange();
206     }
207
208     if(isBraking) {
209         setCCOn(false);
210         return false; //if brake is pressed, exit CC
211     }
212     // Check if the vehicle has reached the desired speed
213     if (sensorFusion.getCurrentSpeed() == desiredCCSpeed) {
214         break; // Exit the loop if the desired speed is reached
215     }
216 }
217
218     return true;
219 }
220
221 bool PlanningVCS::accelerate() { //accelerate the vehicle according to gas pedal intensity
222     if(isAccelerating&&isBraking) return false; //if brake and gas are simultaneously pressed, do not perform acceleration
223     while(isAccelerating){ //while gas pedal is engaged
224         setCurrentSpeed(sensorFusion.getCurrentSpeed() + gasPedalIntensity); //increment speed
225         if(isBraking) return false; //if brake is pressed while accelerating, break
226     }
227     return true;
228 }
229
230 bool PlanningVCS::brake() { //decelerate the vehicle according to brake pedal intensity
231     if(isAccelerating&&isBraking) return false; //if brake and gas are simultaneously pressed, do not perform deceleration
232     while(isBraking){ //while brake pedal is engaged
233         setCurrentSpeed(sensorFusion.getCurrentSpeed() - brakeIntensity); //decrement speed
234         if(isAccelerating) return false; //if gas pedal pressed while braking, break
235     }
236     return true;
237 }
238
239 bool PlanningVCS::accelerateTo(double desiredSpeed) { //accelerate vehicle to a set speed (CC)
240     if (desiredSpeed < sensorFusion.getCurrentSpeed() || desiredSpeed > 90) return false; //desired speed is less than current speed
241     setCurrentSpeed(desiredSpeed);
242     return true;
243 }
244
245 bool PlanningVCS::brakeTo(double desiredSpeed) { //decelerate vehicle to a set speed (CC)
246     if (desiredSpeed > sensorFusion.getCurrentSpeed() || desiredSpeed < 0) return false; //desired speed is greater than current speed or negative
247     setCurrentSpeed(desiredSpeed);
248     return true;
249 }
250

```

```

251 bool PlanningVCS::attemptLaneChange() { //attempt to switch the vehicle's lane
252     if(!sensorFusion.getMarkedRoad()||getTurnSignal()==-1) return false; //if road is not marked or turn signal is not on, do not perform
253     if(getTurnSignal() == 0 && !sensorFusion.getLeftLaneOccupied()){ //left //if trying to move left and left lane is not occupied
254         if (setcurrentLane(sensorFusion.getcurrentLane() - 1)) //update current lane to left lane
255             setsteeringDirection(0); //reset steering direction to straight
256     } else
257         return false;
258 } else if (!sensorFusion.getrightLaneOccupied()){ //right //if trying to move right and right lane is not occupied
259     if (setcurrentLane(sensorFusion.getcurrentLane() + 1)) //update current lane to left lane
260         setsteeringDirection(0); //reset steering direction to straight
261     } else
262         return false;
263 } else
264     return false;
265 return true;
266 }
267
268 bool PlanningVCS::makeTurn(double steeringDirection, double distance) { // Makes a turn with the given steering direction and distance. Steering d
269     if (getTurnSignal() == -1 || steeringDirection == 0) return false; //if given direction is straight or turn signal is not activated, do not pe
270     double temp = 0; //temp variable to help store vehicle position
271     while(temp!=distance && !sensorFusion.getObjectInFront()){ //while vehicle has not reached posotion and no object is detected in front
272         if((steeringDirection<0 && sensorFusion.getLeftLaneOccupied())||(steeringDirection>0 && sensorFusion.getrightLaneOccupied())){ //if the c
273             return false;
274         }
275         setsteeringDirection(steeringDirection); //set the steering direction to given calculated direction
276         temp += (steeringDirection*sensorFusion.getcurrentSpeed()); //update vehicle position according to steering direction and speed
277     }
278     return true;
279 }
280
281 void PlanningVCS::maintainLightStatus(){
282     int status = getheadlights(); //get current status. (off(0), low(1), high(2))
283     double level = sensorFusion.getlightLevel();
284     if(status == 0){ //if theyre off, check if low beams need to go on
285         if(level <= 10) setwipers(1);
286     }
287     else if (status == 1){ //if low beams on, check if need to be turned off or high beams need activated
288         if(level <= 5) setwipers(2);
289         else if (level>=10) setwipers(0);
290     }
291     else if (status == 2){ //if high beams on, check if need to downgrade to low
292         if(level >=5) setwipers(1);
293     }
294 }
295
296 void PlanningVCS::maintainWiperStatus(){
297     int status = setwipers(); //get current status (off, on, auto)(-1,0,1)
298     if(status == 0) return; //confirm regular wipers are not on
299     double level = sensorFusion.getrainAmount();
300
301     if(status == -1){ //if wipers are off, check if enough rain to turn them on
302         if(level>=0.1) setwipers(2);
303     }
304     else if(status == 1){ //if wipers are on(auto), check rain level to see if they need to be turned off
305         if(level<=0.1) setwipers(-1);
306     }
307
308     //if off, check if water level detected to turn on
309     //if auto, check if wipers need to be turned off
310
311 }
312

```

5.3.3 Interaction Interface

```
1  /* interaction.hpp
2     This file contains the class definitions for the SystemManagement and Display classes.
3     These classes are used to manage the system and display information to the user.
4  */
5
6  #ifndef INTERACTION_HPP
7  #define INTERACTION_HPP
8
9  #include <string>
10 #include <map>
11 #include <vector>
12
13 #include "planningVCS.hpp"
14
15 using namespace std;
16
17 class SystemManagement {
18     private:
19
20         PlanningVCS &planningVCS; // PlanningVCS object for accessing data from sensors
21
22         bool loggedIn; // True if a user is logged in, false otherwise.
23
24         string currentUser; // Name of the current user that is logged in
25
26         map<string,string> credentials; // Stores pairs of the form (username, password)
27
28         vector<string> technicians; // Usernames of the technicians
29
30         vector<string> logs; // List of logs from the system
31
32         void init_credentials(); // Initializes the values of credentials and technicians
33
34     public:
35         SystemManagement(PlanningVCS &planningVCS); // Constructor for SystemManagement
36
37         bool getLoggedIn() const; // Return loggedIn
38
39         string getLogs() const; // Return logs delimited by new lines
40
41         bool isTechnician() const; // Returns true if currentUser is a technician
42
43         string getCurrentUser() const; // Return currentUser
44
45         string getStatus() const; // Returns a string with the vehicle's current state
46
47         /* Check if the username and password correspond to an entry in the list.
48            If the credentials are valid, set loggedIn to true and currentUser to
49            username.
50         */
```

```
51     bool login(string user, string password);
52
53     void insertLog(string log);
54
55 };
56
57 class Display {
58     private:
59         PlanningVCS& planningVCS; // Reference to PlanningVCS to fetch vehicle data
60         SystemManagement& systemManagement; // Reference to SystemManagement to fetch system data
61
62     public:
63         // Constructor for Display
64         Display(PlanningVCS& planningVCS, SystemManagement& systemManagement);
65
66         // Clears the console screen
67         void clearScreen();
68
69         // Updates and prints all display components
70         void updateAndPrintDisplay();
71
72         //Sets up an interactive driver display
73         void interactiveDisplay(SensorFusion& sensorFusion);
74
75         //Helps perform actions from user input with switch statement
76         void doAction(int action, Display& display, SensorFusion& sensorFusion );
77 };
78
79 #endif // INTERACTION_HPP
80
```

6. Project Code

6.1 Data Aggregation

```
1  /*dataAggregation.cpp*/
2  #include <sstream>
3  #include <iostream>
4
5  #include "dataAggregation.hpp"
6
7  using namespace std;
8
9  SensorData stringToSensorData(string data) {
10     SensorData sensorData;
11     // Parse a comma delimited string, each entry corresponding to the same field in SensorData
12     // Example: "1,0,2,3,45,0.5,10,5,1,0,1,10,100,0.5"
13     stringstream ss(data);
14     string token;
15     getline(ss, token, ',');
16     sensorData.isOnHighway = (stoi(token) == 1) ? true : false;
17     getline(ss, token, ',');
18     sensorData.markedRoad = (stoi(token) == 1) ? true : false;
19     getline(ss, token, ',');
20     sensorData.currentLane = stoi(token);
21     getline(ss, token, ',');
22     sensorData.nLanes = stoi(token);
23     getline(ss, token, ',');
24     sensorData.currentSpeed = stod(token);
25     getline(ss, token, ',');
26     sensorData.steeringDirection = stod(token);
27     getline(ss, token, ',');
28     sensorData.laneWidth = stod(token);
29     getline(ss, token, ',');
30     sensorData.lanePos = stod(token);
31     getline(ss, token, ',');
32     sensorData.leftLaneOccupied = (stoi(token) == 1) ? true : false;
33     getline(ss, token, ',');
34     sensorData.rightLaneOccupied = (stoi(token) == 1) ? true : false;
35     getline(ss, token, ',');
36     sensorData.objectInFront = (stoi(token) == 1) ? true : false;
37     getline(ss, token, ',');
38     sensorData.objectDistance = stod(token);
39     getline(ss, token, ',');
40     sensorData.lightLevel = stod(token);
41     getline(ss, token, ',');
42     sensorData.rainAmount = stod(token);
43     getline(ss, token, ',');
44     sensorData.temperature = stod(token);
45
46     return sensorData;
47 }
48
49 void printSensorData(SensorData data) {
50     cout << "{":
```

```

51     cout << "isOnHighway = " << (data.isOnHighway ? "true" : "false");
52     cout << ", markedRoad = " << (data.markedRoad ? "true" : "false");
53     cout << ", currentLane = " << data.currentLane;
54     cout << ", nLanes = " << data.nLanes;
55     cout << ", currentSpeed = " << data.currentSpeed;
56     cout << ", steeringDirection = " << data.steeringDirection;
57     cout << ", laneWidth = " << data.laneWidth;
58     cout << ", lanePos = " << data.lanePos;
59     cout << ", leftLaneOccupied = " << (data.leftLaneOccupied ? "true" : "false");
60     cout << ", rightLaneOccupied = " << (data.rightLaneOccupied ? "true" : "false");
61     cout << ", objectInFront = " << (data.objectInFront ? "true" : "false");
62     cout << ", objectDistance = " << data.objectDistance;
63     cout << ", lightLevel = " << data.lightLevel;
64     cout << ", rainAmount = " << data.rainAmount;
65     cout << "}" << endl;
66
67 }
68 /* GPS CLASS IMPLEMENTATIONS
69 */
70 GPS::GPS() {
71     isOnHighway = false;
72     markedRoad = false;
73     currentLane = 0;
74     nLanes = 1;
75 }
76
77 bool GPS::getisOnHighway() const {
78     return isOnHighway;
79 }
80
81 bool GPS::getmarkedRoad() const {
82     return markedRoad;
83 }
84
85 int GPS::getcurrentLane() const {
86     return currentLane;
87 }
88
89 int GPS::getnLanes() const {
90     return nLanes;
91 }
92
93
94 void GPS::update(SensorData sensorData) {
95     isOnHighway = sensorData.isOnHighway;
96     markedRoad = sensorData.markedRoad;
97     currentLane = sensorData.currentLane;
98     nLanes = sensorData.nLanes;
99 }
100

```

```
101  /* IMU CLASS IMPLEMENTATIONS
102  */
103
104  IMU::IMU() {
105      currentSpeed = 0;
106      steeringDirection = 0;
107  }
108
109  double IMU::getCurrentSpeed() const {
110      return currentSpeed;
111  }
112
113  double IMU::getsteeringDirection() const {
114      return steeringDirection;
115  }
116
117  void IMU::update(SensorData sensorData) {
118      // Updates the private state of IMU with current vehicle data
119      currentSpeed = sensorData.currentSpeed;
120      steeringDirection = sensorData.steeringDirection;
121  }
122
123  /* PERCEPTION CLASS IMPLEMENTATIONS
124  */
125
126  Perception::Perception() {
127      laneWidth = 0;
128      lanePos = 0;
129      leftLaneOccupied = false;
130      rightLaneOccupied = false;
131      objectInFront = false;
132      objectDistance = __DBL_MAX__;
133      lightLevel = 100; // ARBITRARY VALUE
134      rainAmount = 0;
135  }
136
137  double Perception::getlaneWidth() const {
138      return laneWidth;
139  }
140
141  double Perception::getlanePos() const {
142      return lanePos;
143  }
144
145  bool Perception::getleftLaneOccupied() const {
146      return leftLaneOccupied;
147  }
148
149  bool Perception::getrightLaneOccupied() const {
150      return rightLaneOccupied;
```



```

151 }
152
153 bool Perception::getobjectInFront() const {
154     return objectInFront;
155 }
156
157 double Perception::getobjectDistance() const {
158     return objectDistance;
159 }
160
161 double Perception::getlightLevel() const {
162     return lightLevel;
163 }
164
165 double Perception::getrainAmount() const {
166     return rainAmount;
167 }
168
169 void Perception::update(SensorData sensorData) {
170     laneWidth = sensorData.laneWidth;
171     lanePos = sensorData.lanePos;
172     leftLaneOccupied = sensorData.leftLaneOccupied;
173     rightLaneOccupied = sensorData.rightLaneOccupied;
174     objectInFront = sensorData.objectInFront;
175     objectDistance = sensorData.objectDistance;
176     lightLevel = sensorData.lightLevel;
177     rainAmount = sensorData.rainAmount;
178 }
179
180 /* SENSORFUSION CLASS IMPLEMENTATIONS
181 */
182
183 SensorFusion::SensorFusion() {
184     gps = GPS();
185     imu = IMU();
186     sensors = Perception();
187 ;
188 }
189
190 bool SensorFusion::getisOnHighway() const {
191     return gps.getisOnHighway();
192 }
193
194 bool SensorFusion::getmarkedRoad() const {
195     return gps.getmarkedRoad();
196 }
197
198 int SensorFusion::getcurrentLane() const {
199     return gps.getcurrentLane();
200 }

```

```
201
202 int SensorFusion::getnLanes() const {
203     return gps.getnLanes();
204 }
205
206 double SensorFusion::getcurrentSpeed() const {
207     return imu.getcurrentSpeed();
208 }
209
210 double SensorFusion::getsteeringDirection() const {
211     return imu.getsteeringDirection();
212 }
213
214 double SensorFusion::getlaneWidth() const {
215     return sensors.getlaneWidth();
216 }
217
218 double SensorFusion::getlanePos() const {
219     return sensors.getlanePos();
220 }
221
222 bool SensorFusion::getleftLaneOccupied() const {
223     return sensors.getleftLaneOccupied();
224 }
225
226 bool SensorFusion::getrightLaneOccupied() const {
227     return sensors.getrightLaneOccupied();
228 }
229
230 bool SensorFusion::getobjectInFront() const {
231     return sensors.getobjectInFront();
232 }
233
234 double SensorFusion::getobjectDistance() const {
235     return sensors.getobjectDistance();
236 }
237
238 double SensorFusion::getlightLevel() const {
239     return sensors.getlightLevel();
240 }
241
242 double SensorFusion::getrainAmount() const {
243     return sensors.getrainAmount();
244 }
245
246 bool SensorFusion::setcurrentSpeed(double speed) {
247     imu.currentSpeed = speed;
248     return true;
249 }
250
```

```
251 bool SensorFusion::setsteeringDirection(double direction) {
252     imu.steeringDirection = direction;
253     return true;
254 }
255
256 bool SensorFusion::setcurrentLane(int lane) {
257     gps.currentLane = lane;
258     return true;
259 }
260
261 void SensorFusion::update(SensorData sensorData) {
262     gps.update(sensorData);
263     imu.update(sensorData);
264     sensors.update(sensorData);
265 }
```

6.2 Planning-VCS

```
1  /*planningvcs.cpp*/
2  #include <iostream>
3  #include "planningVCS.hpp"
4  #include "dataAggregation.hpp"
5
6  #include "interaction.hpp"
7  PlanningVCS::PlanningVCS(SensorFusion &sensorFusion) //set default values
8      : sensorFusion(sensorFusion) {
9      wipers = -1; //wipers off
10     headlights = 0; //headlights off
11     temperature = 72; //default temperature is 72 degrees
12     turnSignal = -1; //turn signal neutral
13     ccOn = false; //Cruise Control not on
14     gear = 0; //gear in park
15     isBraking = false; //not braking
16     isAccelerating = false; //not accelerating
17     ccSpeed = 60; // Default speed for cruise control is 60 mph
18     hazardLightsOn = false; // the lights are not on
19 }
20
21
22 int PlanningVCS::getwipers() const { //get status of wipers, -1 is off, 1 is on, 2 is on and in auto mode
23     return wipers;
24 }
25
26 double PlanningVCS::getTemperature() const {
27     return temperature;
28 }
29
30 int PlanningVCS::getheadlights() const { //get headlight status, 0 if off, 1 if low beams on, 2 if high beams on
31     return headlights;
32 }
33
34 int PlanningVCS::getturnSignal() const { //get turn signal status. -1 off, 0 left, 1 right
35     return turnSignal;
36 }
37
38 bool PlanningVCS::getccOn() const { //get Cruise Control status. true is on
39     return ccOn;
40 }
41
42 int PlanningVCS::getgear() const { //return gear int. 0 park, 1 drive, 2 neutral, 3 drive
43     return gear;
44 }
45
46 bool PlanningVCS::getisBraking() const { //Returns true if vehicle is braking
47     return isBraking;
48 }
49
50
```

```

51 bool PlanningVCS::getisAccelerating() const { //Returns true if vehicle is accelerating
52     return isAccelerating;
53 }
54 bool PlanningVCS::getHazardLightsStatus() const { //return true if lights are on
55     return hazardLightsOn;
56 }
57
58 SensorFusion& PlanningVCS::getsensorFusion() const { //Returns sensor data
59     return sensorFusion;
60 }
61
62 double PlanningVCS::getccSpeed() const { // Returns ccSpeed
63     return ccSpeed;
64 }
65
66 bool PlanningVCS::setwipers(int level) { //set wipers in range form -1 to 1
67     if (level >= -1 && level <= 1) {
68         wipers = level;
69         return true;
70     }
71     return false;
72 }
73
74
75 bool PlanningVCS::setheadlights(int level) { //set headlights in range from 0 to 2
76     if (level >= 0 && level <= 2) {
77         headlights = level;
78         return true;
79     }
80     return false;
81 }
82
83 bool PlanningVCS::setcurrentLane(int lane) { //set current lane in range from 0 to nLanes - 1
84     if (lane >= 0 && lane < sensorFusion.getnLanes()) {
85         sensorFusion.setcurrentLane(lane);
86         return true;
87     }
88     return false;
89 }
90
91 bool PlanningVCS::setturnSignal(int level) { //set turn signal in range from -1 to 1
92     if (level >= -1 && level <= 1) {
93         turnSignal = level;
94         return true;
95     }
96     return false;
97 }
98
99 bool PlanningVCS::setccOn(bool b) { //set the mode for cruise control, on or off
100     ccOn = b;

```

```

101     return true;
102 }
103
104 bool PlanningVCS::setccSpeed(double speed) { // Stes the ccSpeed between 50 and 70
105     if (speed < 50 || speed > 70) return false;
106     ccSpeed = speed;
107     return true;
108 }
109
110 bool PlanningVCS::setgear(int level) { //set the gear level in range from 0 to 3
111     if (level >= 0 && level <= 3) {
112         gear = level;
113         return true;
114     }
115     return false;
116 }
117
118 bool PlanningVCS::setisBraking(bool b) { //set isBraking to true or false
119     isBraking = b;
120     return true;
121 }
122
123 bool PlanningVCS::setisAccelerating(bool b) { //set isAccelerating to true or false
124     isAccelerating = b;
125     return true;
126 }
127
128 bool PlanningVCS::setcurrentSpeed(double speed) {
129     if (speed > 110) {
130         cout << "Error: Speed limit exceeded. Setting speed to 70 mph." << endl;
131         sensorFusion.setcurrentSpeed(70);
132         return true; // Indicating that the speed has been set, albeit not to the requested speed
133     }
134     return sensorFusion.setcurrentSpeed(speed);
135 }
136
137 bool PlanningVCS::setsteeringDirection(double direction) {
138     return sensorFusion.setsteeringDirection(direction);
139 }
140 void PlanningVCS::toggleHazardLights() {
141     hazardLightsOn = !hazardLightsOn;
142 }
143
144 void PlanningVCS::setTemperature(double temp) {
145     if (temp >= 50 && temp <= 80) {
146         temperature = temp;
147     } else {
148         cout << "Invalid temperature. Please choose a value between 50-80 degrees." << endl;
149     }
150 }

```

```

151 void PlanningVCS::update(SensorData sensorData) {
152     sensorFusion.update(sensorData);
153     // Check if the vehicle is on a highway
154     if (sensorData.currentSpeed != 0) {
155         setgear(1);
156     } else {
157         setgear(0);
158     }
159     if (!sensorFusion.getisOnHighway()) {
160         // If the vehicle is not on a highway, ensure cruise control is turned off
161         setccOn(false);
162     } else {
163         // If the vehicle is on a highway, proceed with cruise control logic
164         if (ccOn && getturnSignal() == -1) {
165             // If cruise control is on and no turn signal is active, manage speed
166             double currentSpeed = sensorFusion.getCurrentSpeed();
167             double desiredSpeed = getccSpeed(); // Assuming there is a method to fetch the desired speed
168
169             if (currentSpeed < desiredSpeed) {
170                 accelerateTo(desiredSpeed);
171             } else if (currentSpeed > desiredSpeed) {
172                 brakeTo(desiredSpeed);
173             }
174         } else if (ccOn && (getturnSignal() == 0 || getturnSignal() == 1)) {
175             // If the turn signal is on, attempt to change lanes
176             attemptLaneChange();
177         }
178     }
179 }
180 if (sensorFusion.getlightLevel() <= HIGHBEAM_THRESHOLD)
181     setheadlights(2);
182 else if (sensorFusion.getlightLevel() <= LOWBEAM_THRESHOLD)
183     setheadlights(1);
184 else
185     setheadlights(0);
186 if (sensorFusion.getrainAmount() >= RAIN_THRESHOLD)
187     setwipers(1);
188 else
189     setwipers(-1);
190 temperature = sensorData.temperature;
191 }
192
193
194 bool PlanningVCS::doCC(double desiredCCSpeed) { //Initiate cruise control
195     if(desiredCCSpeed > 90 || desiredCCSpeed < 25 || !sensorFusion.getisOnHighway() || !getturnSignal()) return false; //check if speed is within sa
196     while(ccOn && sensorFusion.getmarkedRoad()){ //double check cc is on and the current road is marked
197         //decide whether to accelerate/decelerate the vehicle to desired speed
198         if(sensorFusion.getCurrentSpeed() < desiredCCSpeed)
199             {accelerateTo(desiredCCSpeed); }
200         else {

```

```

201         brakeTo(desiredCCSpeed);
202     }
203     //if turn signal is activated, attempt to change lanes
204     if(getTurnSignal() == 0 || getTurnSignal() == 1){
205         attemptLaneChange();
206     }
207
208     if(isBraking) {
209         setCCOn(false);
210         return false; //if brake is pressed, exit CC
211     }
212     // Check if the vehicle has reached the desired speed
213     if (sensorFusion.getCurrentSpeed() == desiredCCSpeed) {
214         break; // Exit the loop if the desired speed is reached
215     }
216 }
217
218     return true;
219 }
220
221 bool PlanningVCS::accelerate() { //accelerate the vehicle according to gas pedal intensity
222     if(isAccelerating&&isBraking) return false; //if brake and gas are simultaneously pressed, do not perform acceleration
223     while(isAccelerating){ //while gas pedal is engaged
224         setCurrentSpeed(sensorFusion.getCurrentSpeed() + gasPedalIntensity); //increment speed
225         if(isBraking) return false; //if brake is pressed while accelerating, break
226     }
227     return true;
228 }
229
230 bool PlanningVCS::brake() { //decelerate the vehicle according to brake pedal intensity
231     if(isAccelerating&&isBraking) return false; //if brake and gas are simultaneously pressed, do not perform deceleration
232     while(isBraking){ //while brake pedal is engaged
233         setCurrentSpeed(sensorFusion.getCurrentSpeed() - brakeIntensity); //decrement speed
234         if(isAccelerating) return false; //if gas pedal pressed while braking, break
235     }
236     return true;
237 }
238
239 bool PlanningVCS::accelerateTo(double desiredSpeed) { //accelerate vehicle to a set speed (CC)
240     if (desiredSpeed < sensorFusion.getCurrentSpeed() || desiredSpeed > 90) return false; //desired speed is less than current speed
241     setCurrentSpeed(desiredSpeed);
242     return true;
243 }
244
245 bool PlanningVCS::brakeTo(double desiredSpeed) { //decelerate vehicle to a set speed (CC)
246     if (desiredSpeed > sensorFusion.getCurrentSpeed() || desiredSpeed < 0) return false; //desired speed is greater than current speed or negative
247     setCurrentSpeed(desiredSpeed);
248     return true;
249 }
250

```



```

251 bool PlanningVCS::attemptLaneChange() { //attempt to switch the vehicle's lane
252     if(!sensorFusion.getMarkedRoad()||getTurnSignal()==-1) return false; //if road is not marked or turn signal is not on, do not perform
253     if(getTurnSignal() == 0 && !sensorFusion.getLeftLaneOccupied()){ //left //if trying to move left and left lane is not occupied
254         if (setCurrentLane(sensorFusion.getCurrentLane() - 1)) //update current lane to left lane
255             setsteeringDirection(0); //reset steering direction to straight
256         else
257             return false;
258     } else if (!sensorFusion.getRightLaneOccupied()){ //right //if trying to move right and right lane is not occupied
259         if (setCurrentLane(sensorFusion.getCurrentLane() + 1)) //update current lane to left lane
260             setsteeringDirection(0); //reset steering direction to straight
261         else
262             return false;
263     } else
264         return false;
265     return true;
266 }
267
268 bool PlanningVCS::makeTurn(double steeringDirection, double distance) { // Makes a turn with the given steering direction and distance. Steering d
269     if (getTurnSignal() == -1 || steeringDirection == 0) return false; //if given direction is straight or turn signal is not activated, do not pe
270     double temp = 0; //temp variable to help store vehicle position
271     while(temp!=distance && !sensorFusion.getObjectInFront()){ //while vehicle has not reached posotion and no object is detected in front
272         if((steeringDirection<0 && sensorFusion.getLeftLaneOccupied())||(steeringDirection>0 && sensorFusion.getRightLaneOccupied())){ //if the c
273             return false;
274         }
275         setsteeringDirection(steeringDirection); //set the steering direction to given calculated direction
276         temp += (steeringDirection*sensorFusion.getCurrentSpeed()); //update vehicle position according to steering direction and speed
277     }
278     return true;
279 }
280
281
282 void PlanningVCS::maintainLightStatus(){
283     int status = getHeadlights(); //get current status. (off(0), low(1), high(2))
284     double level = sensorFusion.getLightLevel();
285     if(status == 0){ //if they're off, check if low beams need to go on
286         if(level <= 10) setwipers(1);
287     }
288     else if (status == 1){ //if low beams on, check if need to be turned off or high beams need activated
289         if(level <= 5) setwipers(2);
290         else if (level>=10) setwipers(0);
291     }
292     else if (status == 2){ //if high beams on, check if need to downgrade to low
293         if(level >=5) setwipers(1);
294     }
295 }
296
297 void PlanningVCS::maintainWiperStatus(){
298     int status = getwipers(); //get current status (off, on, auto)(-1,0,1)
299     if(status == 0) return; //confirm regular wipers are not on
300     double level = sensorFusion.getRainAmount();
301
302     if(status == -1){ //if wipers are off, check if enough rain to turn them on
303         if(level>=0.1) setwipers(2);
304     }
305     else if(status == 1){ //if wipers are on(auto), check rain level to see if they need to be turned off
306         if(level<=0.1) setwipers(-1);
307     }
308
309     //if off, check if water level detected to turn on
310     //if auto, check if wipers need to be turned off
311 }
312

```

6.3 Interaction

```
1  /*interactions.cpp*/
2  #include <iostream>
3  #include <algorithm>
4  #include <sstream>
5  #include <cstdlib>
6  #include <chrono>
7  #include <thread>
8  #include <unistd.h>
9
10
11 #include "interaction.hpp"
12 #include "planningVCS.hpp"
13 #include "dataAggregation.hpp"
14 using namespace std;
15
16 SystemManagement::SystemManagement(PlanningVCS &planningVCS)
17     : planningVCS(planningVCS) {
18     loggedIn = false;
19     currentUser = "";
20     init_credentials();
21 }
22
23 bool SystemManagement::getLoggedIn() const {
24     return loggedIn;
25 }
26
27 bool SystemManagement::isTechnician() const {
28     return find(technicians.begin(), technicians.end(), currentUser) != technicians.end();
29 }
30
31 string SystemManagement::getCurrentUser() const {
32     return currentUser;
33 }
34
35 bool SystemManagement::login(string user, string password) {
36     if (credentials.find(user) != credentials.end() && credentials[user] == password) {
37         loggedIn = true;
38         currentUser = user;
39         return true;
40     }
41     return false;
42 }
43
44 string SystemManagement::getLogs() const {
45     if (!isTechnician()) return "";
46     stringstream builder;
47     for (const auto& log : logs) {
48         builder << log << endl;
49     }
50     return builder.str();
51 }
```

```

51 }
52
53 string SystemManagement::getStatus() const {
54     if (!isTechnician()) return "";
55     stringstream builder;
56     SensorFusion& sensorfusion = planningVCS.getsensorFusion();
57     builder << "On Highway: " << (sensorfusion.getisOnHighway() ? "True" : "False") << endl;
58     builder << "Road is Marked: " << (sensorfusion.getmarkedRoad() ? "True" : "False") << endl;
59     builder << "Left Lane Occupied: " << (sensorfusion.getleftLaneOccupied() ? "True" : "False") << endl;
60     builder << "Right Lane Occupied: " << (sensorfusion.getrightLaneOccupied() ? "True" : "False") << endl;
61     builder << "Object in Front of Vehicle: " << (sensorfusion.getobjectInFront() ? "True" : "False") << endl;
62     builder << "Current Lane: " << sensorfusion.getcurrentLane() << endl;
63     builder << "Number of Lanes: " << sensorfusion.getnLanes() << endl;
64     builder << "Current Speed (MPH): " << sensorfusion.getcurrentSpeed() << endl;
65     builder << "Steering Direction: " << sensorfusion.getsteeringDirection() << endl;
66     builder << "Lane Width (ft): " << sensorfusion.getlaneWidth() << endl;
67     builder << "Lane Position (ft from left line): " << sensorfusion.getlanePos() << endl;
68     builder << "Object Distance (ft): " << sensorfusion.getobjectDistance() << endl;
69     builder << "Light Level: " << sensorfusion.getlightLevel() << endl;
70     builder << "Rain Amount: " << sensorfusion.getrainAmount() << endl;
71     builder << "Hazard Lights: " << (planningVCS.getHazardLightsStatus() ? "On" : "Off") << endl;
72     builder << "Temperature(°F): " << planningVCS.getTemperature() << endl;
73
74     switch(planningVCS.getwipers()) {
75     case -1:
76         builder << "Wipers: Off" << endl;
77         break;
78     case 0:
79         builder << "Wipers: On" << endl;
80         break;
81     case 1:
82         builder << "Wipers: Auto" << endl;
83         break;
84     }
85     switch(planningVCS.getheadlights()) {
86     case 0:
87         builder << "Headlights: Off" << endl;
88         break;
89     case 1:
90         builder << "Headlights: Low Beams" << endl;
91         break;
92     case 2:
93         builder << "Headlights: High Beams" << endl;
94         break;
95     }
96     switch(planningVCS.getturnSignal()) {
97     case -1:
98         builder << "Turn Signal: Off" << endl;
99         break;
100    case 0:

```

```

101         builder << "Turn Signal: Left" << endl;
102         break;
103     case 1:
104         builder << "Turn Signal: Right" << endl;
105         break;
106     }
107     builder << "Cruise Control: " << (planningVCS.getccOn() ? "On" : "Off") << endl;
108     switch(planningVCS.getgear()) {
109     case 0:
110         builder << "Gear: Park" << endl;
111         break;
112     case 1:
113         builder << "Gear: Drive" << endl;
114         break;
115     case 2:
116         builder << "Gear: Neutral" << endl;
117         break;
118     case 3:
119         builder << "Gear: Reverse" << endl;
120         break;
121     }
122     builder << "Braking: " << (planningVCS.getisBraking() ? "True" : "False") << endl;
123     builder << "Accelerating: " << (planningVCS.getisAccelerating() ? "True" : "False") << endl;
124     return builder.str();
125 }
126
127
128 void SystemManagement::insertLog(string log) {
129     stringstream builder;
130     builder << logs.size() + 1 << ". " << currentUser << " - " << log;
131     logs.push_back(builder.str());
132 }
133
134 void SystemManagement::init_credentials() {
135     credentials["user"] = "password";
136     credentials["technician"] = "password";
137     technicians.push_back("technician");
138 }
139
140
141
142 // Constructor initializing the reference to PlanningVCS
143 Display::Display(PlanningVCS& vcs, SystemManagement& systemManagement)
144     : planningVCS(vcs), systemManagement(systemManagement) {}
145
146 // Clears the console screen
147 void Display::clearScreen() {
148     // ANSI escape codes to clear the console and reset cursor position
149     std::cout << "\033[2J\033[H";
150 }

```

```

151
152 // Updates and prints all display components
153 /*
154     want to display:
155         turn signals-
156         headlight levels-
157         wiper status-
158         current gear-
159         current velocity-
160         cruise control status-
161         camera? gas levels? external temp? warning system? total mileage?
162 */
163 void Display::updateAndPrintDisplay() {
164     std::cout << "Vehicle Dashboard:" << std::endl;
165     std::cout << "-----" << std::endl;
166
167     // Update display values from the PlanningVCS instance
168     SensorFusion &sensorFusion = planningVCS.getsensorFusion();
169
170     // Print the updated values
171     std::cout << "Speed: " << sensorFusion.getCurrentSpeed() << " mph" << std::endl;
172     switch (planningVCS.getheadlights()) {
173     case 0:
174         std::cout << "Headlights: Off" << std::endl;
175         break;
176     case 1:
177         std::cout << "Headlights: Low Beams" << std::endl;
178         break;
179     case 2:
180         std::cout << "Headlights: High Beams" << std::endl;
181         break;
182     }
183     switch (planningVCS.getturnSignal()) {
184     case -1:
185         std::cout << "Turn Signal: Off" << std::endl;
186         break;
187     case 0:
188         std::cout << "Turn Signal: Left" << std::endl;
189         break;
190     case 1:
191         std::cout << "Turn Signal: Right" << std::endl;
192         break;
193     }
194
195     switch (planningVCS.getwipers()) {
196     case -1:
197         std::cout << "Wipers: Off" << std::endl;
198         break;
199     case 0:
200         std::cout << "Wipers: On" << std::endl;

```

```

201         break;
202     case 1:
203         std::cout << "Wipers: Auto" << std::endl;
204         break;
205 }
206
207 switch (planningVCS.getgear()) {
208     case 0:
209         std::cout << "Gear: Park" << std::endl;
210         break;
211     case 1:
212         std::cout << "Gear: Drive" << std::endl;
213         break;
214     case 2:
215         std::cout << "Gear: Neutral" << std::endl;
216         break;
217     case 3:
218         std::cout << "Gear: Reverse" << std::endl;
219 }
220 cout<<"Current Lane: " << sensorFusion.getcurrentLane() << std::endl;
221 string temp = "OFF";
222 if(planningVCS.getccOn()) temp = "ON";
223 cout<< "Cruise Control: " << temp << std::endl;
224 cout << "Interior Temperature: " << planningVCS.getTemperature() << " °F" << endl;
225 cout << "Hazard Lights: " << (planningVCS.getHazardLightsStatus() ? "ON" : "OFF") << endl;
226 }
227
228 void Display::interactiveDisplay(SensorFusion& sensorFusion){
229     system("clear");
230     Display display(planningVCS, systemManagement);
231     display.updateAndPrintDisplay();
232     int input;
233     cout<<"(1) Toggle Cruise Control (0) Quit"<<endl;
234     cin>>input;
235     if(input == 1){
236         int dSpeed;
237         if (sensorFusion.getcurrentSpeed() < 50) {
238             cout << "Cannot toggle cruise control when the vehicle is not moving slower than 50mph." << endl;
239             systemManagement.insertLog("Cruise control cannot be toggled when the vehicle is not moving fast enough");
240             return;
241         }
242         if (sensorFusion.getcurrentSpeed() > 125 )
243             cout << "Cannot toggle cruise control when the vehicle is not moving faster than than 125mph." << endl;
244             systemManagement.insertLog("Cruise control cannot be toggled when the vehicle is moving too fast enough");
245             return;
246         }
247         if(planningVCS.getgear()!=1){
248             cout << "Vehicle must be in drive to engage in cruise control." << endl;
249             systemManagement.insertLog("Cruise control cannot be toggled when the vehicle is not in drive.");
250             return;
251         }
252     }

```

```

251     cout<<"Input Desired Speed: ";
252     cin>>dSpeed;
253     if(dSpeed>=50){
254         planningVCS.setccOn(true);
255         planningVCS.setcurrentSpeed(dSpeed);
256         system("clear");
257         display.updateAndPrintDisplay();
258         cout<<" (2) Brake \n (3) Accelerate \n (4) Change Headlights \n (5) Change Lanes \n (6) Set Temperature \n (7) Toggle Hazard Lights \n
259         cin>>input;
260         display.doAction(input, display, sensorFusion);
261     }
262     else{
263         cout<<"Cruise Control Speed must be 50 MPH or faster."<<endl;
264         return;
265     }
266
267 }
268 else return;
269
270
271
272
273 }
274 void Display::doAction(int action, Display& display, SensorFusion& sensorFusion){
275     while(true){
276         double speed, currentSpeed;
277         switch (action) {
278             case 0:
279                 cout<<"Turning CC off. Exiting..."<<endl;
280                 planningVCS.setccOn(false);
281                 sleep(2);
282                 return;
283                 break;
284             case 2:
285                 cout << "Enter desired speed: ";
286                 cin >> speed;
287                 currentSpeed = planningVCS.getsensorFusion().getcurrentSpeed();
288                 if (speed < 0 || speed > currentSpeed) {
289                     cout << "Invalid speed. Must be between 0 and " << currentSpeed << "." << endl;
290                     systemManagement.insertLog("Invalid speed entered for braking");
291                     break;
292                 }
293                 planningVCS.setcurrentSpeed(speed);
294                 system("clear");
295                 display.updateAndPrintDisplay();
296                 if (planningVCS.getccOn()) {
297                     planningVCS.setccOn(false);
298                     cout << "Cruise control turned off." << endl;
299                     systemManagement.insertLog("Cruise control turned off due to manual braking");
300                 }

```

```

301         cout << "Braking to " << speed << " MPH." << endl;
302         systemManagement.insertLog("Braking to " + to_string(speed) + " MPH");
303         if(speed == 0) planningVCS.setgear(0);
304         sleep(2);
305         return;
306         break;
307     case 3:
308         cout << "Enter desired speed: ";
309         cin >> speed;
310         currentSpeed = planningVCS.getsensorFusion().getcurrentSpeed();
311         if (speed < currentSpeed || speed > 100) {
312             cout << "Invalid speed. Must be between " << currentSpeed << " and 100." << endl;
313             systemManagement.insertLog("Invalid speed entered for accelerating");
314             break;
315         }
316         planningVCS.setcurrentSpeed(speed);
317         system("clear");
318         display.updateAndPrintDisplay();
319         if (planningVCS.getccOn()) {
320             planningVCS.setccOn(false);
321             cout << "Cruise control turned off." << endl;
322             systemManagement.insertLog("Cruise control turned off due to manual accelerating");
323         }
324         cout << "Accelerating to " << speed << " MPH." << endl;
325         systemManagement.insertLog("Braking to " + to_string(speed) + " MPH");
326         sleep(2);
327         return;
328         break;
329     case 4:
330         int newHeadlightStatus;
331         cout << "Enter new headlight status (0: Off, 1: Low, 2: High): ";
332         cin >> newHeadlightStatus;
333         if (planningVCS.setheadlights(newHeadlightStatus)) {
334             cout << "Adjusted headlights" << endl;
335             systemManagement.insertLog("Headlights adjusted successfully");
336             sleep(2);
337         } else {
338             cout << "Invalid input. Failed to adjust headlights." << endl;
339             systemManagement.insertLog("Failed to adjust headlights due to invalid input");
340             sleep(2);
341         }
342     }
343
344     break;
345     case 5:
346         int turnDirection;
347         cout << "(1) Left (2) Right: " << endl;
348         cin >> turnDirection;
349         planningVCS.setturnSignal(turnDirection-1);
350         system("clear");

```



```

351         display.updateAndPrintDisplay();
352         cout << "Press Enter to Confirm" << endl;
353         cin.ignore();
354         cin.get();
355         if (!planningVCS.setturnSignal(turnDirection - 1)) {
356             cout << "Invalid turn signal.\n" << endl;
357             systemManagement.insertLog("Failed to update turn signal");
358             sleep(2);
359         } else {
360             systemManagement.insertLog("Turn signal changed.");
361             if (planningVCS.attemptLaneChange()) {
362                 cout << "Successfully changed lanes." << endl;
363                 systemManagement.insertLog("Changed lanes successfully");
364                 sleep(2);
365             } else {
366                 cout << "!!! Could not change lanes !!!" << endl;
367                 systemManagement.insertLog("Failed to change lanes");
368                 sleep(2);
369             }
370         }
371         planningVCS.setturnSignal(-1);
372         break;
373     case 6:
374         int temp;
375         cout << "Enter desired temperature (50-80): ";
376         cin >> temp;
377         planningVCS.setTemperature(temp);
378         systemManagement.insertLog("Temperature set to " + to_string(temp) + " degrees Fahrenheit");
379         sleep(2);
380         break;
381     case 7:
382         planningVCS.toggleHazardLights();
383         cout << "Hazard lights turned " << (planningVCS.getHazardLightsStatus() ? "on" : "off") << "." << endl;
384         systemManagement.insertLog("Hazard lights toggled");
385         sleep(2);
386         break;
387
388     default:
389         cout << "Invalid action code." << endl;
390         sleep(2);
391         break;
392 }
393 system("clear");
394 display.updateAndPrintDisplay();
395 cout<<" (2) Brake \n (3) Accelerate \n (4) Change Headlights \n (5) Change Lanes \n (6) Set Temperature \n (7) Toggle Hazard L
396 cin>>action;
397 }
398
399 }
400

```

6.3.1 Main

```
1  /*main.cpp*/
2  #include <iostream>
3  #include <fstream>
4  #include <limits>
5
6
7  #include "dataAggregation.hpp"
8  #include "planningVCS.hpp"
9  #include "interaction.hpp"
10
11
12  using namespace std;
13
14
15  int main() {
16      ifstream dataFile;
17      dataFile.open("data.txt");
18      if (!dataFile.is_open()) {
19          cout << "Error opening file" << endl;
20          return 1;
21      }
22
23
24      SensorFusion sensorFusion;
25      PlanningVCS planningVCS(sensorFusion);
26      SystemManagement systemManagement(planningVCS);
27      Display display(planningVCS, systemManagement);
28
29
30      cout << "Welcome to IoT Hug the Lanes!" << endl;
31      cout << "Press Enter to Continue";
32      cin.ignore(numeric_limits<streamsize>::max(), '\n');
33      while (true) {
34          display.clearScreen();
35          string user, pass;
36          cout << "Enter Username: ";
37          cin >> user;
38          cout << "Enter Password: ";
39          cin >> pass;
40          if (systemManagement.login(user, pass)) {
41              cout << "Login Successful" << endl;
42              cin.ignore(numeric_limits<streamsize>::max(), '\n');
43              break;
44          } else {
45              cout << "Login Failed. Press Enter to try again. ";
46              cin.get();
47              cin.ignore(numeric_limits<streamsize>::max(), '\n');
48          }
49      }
50      string line, tmp;
```

```

51 SensorData data;
52 while (true) {
53     cout << "(1) Read Data (2) Print Logs (3) Perform Action (4) Get Vehicle Status (5) Driver Display (6) Logout: ";
54     string actionInput;
55     cin >> actionInput;
56
57     int action;
58     try {
59         action = stoi(actionInput); // Convert string to integer
60     } catch (const invalid_argument& e) {
61         cout << "Invalid action code." << endl;
62         continue; // Skip to the next iteration of the loop
63     }
64     switch (action) {
65         case 1:
66             dataFile.seekg(0, ios::beg);
67             if (!getline(dataFile, line)) {
68                 cout << "Error reading file or EOF" << endl;
69                 systemManagement.insertLog("Error reading data or EOF reached");
70             } else {
71                 data = stringToSensorData(line);
72                 printSensorData(data);
73                 planningVCS.update(data);
74                 // Based on update, print any actions that were performed automatically
75                 systemManagement.insertLog("Data read occurred successfully");
76             }
77             break;
78         case 2:
79             tmp = systemManagement.getlogs();
80             if (tmp.empty()) {
81                 cout << "No logs available or the user is not a technician." << endl;
82                 systemManagement.insertLog("Invalid log request or the user is not a technician");
83             } else {
84                 cout << tmp << endl;
85                 systemManagement.insertLog("Logs accessed successfully");
86             }
87             break;
88         case 3:
89             cout << "Performing Action:" << endl;
90             cout << " (1) Toggle CC \n (2) Brake \n (3) Accelerate \n (4) Change HeadLights \n (5) Change Lanes \n (6) Set Temperature \n (7)
91             int action;
92             double speed, currentSpeed;
93             cin >> action;
94             switch (action) {
95                 case 1:
96                     if (sensorFusion.getCurrentSpeed() < 50) {
97                         cout << "Cannot toggle cruise control when the vehicle is not moving slower than 50mph." << endl;
98                         systemManagement.insertLog("Cruise control cannot be toggled when the vehicle is not moving fast enough");
99                         break;
100             }

```

```

101         else if(planningVCS.getgear() !=1){
102             cout << "Vehicle must be in drive to engage in cruise control." << endl;
103             systemManagement.insertLog("Cruise control cannot be toggled when the vehicle is not in drive.");
104             break;
105         }
106         planningVCS.setccOn(!planningVCS.getccOn());
107         cout << "Cruise control toggled to " << ((planningVCS.getccOn()) ? "ON." : "OFF.") << endl;
108         systemManagement.insertLog("Cruise control successfully toggled.");
109         break;
110     case 2:
111         cout << "Enter desired speed: ";
112         cin >> speed;
113         currentSpeed = planningVCS.getsensorFusion().getcurrentSpeed();
114         if (speed < 0 || speed > currentSpeed) {
115             cout << "Invalid speed. Must be between 0 and " << currentSpeed << "." << endl;
116             systemManagement.insertLog("Invalid speed entered for braking");
117             break;
118         }
119         planningVCS.setcurrentSpeed(speed);
120         if (planningVCS.getccOn()) {
121             planningVCS.setccOn(false);
122             cout << "Cruise control turned off." << endl;
123             systemManagement.insertLog("Cruise control turned off due to manual braking");
124         }
125         cout << "Braking to " << speed << " MPH." << endl;
126         systemManagement.insertLog("Braking to " + to_string(speed) + " MPH");
127         break;
128     case 3:
129         cout << "Enter desired speed: ";
130         cin >> speed;
131         currentSpeed = planningVCS.getsensorFusion().getcurrentSpeed();
132         if (speed < currentSpeed || speed > 100) {
133             cout << "Emergency Break activated: Car must be between " << currentSpeed << " and 100. Car set to " << currentSpeed <
134             systemManagement.insertLog("Emergency Break Initiated: Invalid speed entered for accelerating");
135             break;
136         }
137         if(currentSpeed == 0) planningVCS.setgear(1);
138         planningVCS.setcurrentSpeed(speed);
139         if (planningVCS.getccOn()) {
140             planningVCS.setccOn(false);
141             cout << "Cruise control turned off." << endl;
142             systemManagement.insertLog("Cruise control turned off due to manual accelerating");
143         }
144         if(speed == 0) planningVCS.setgear(0);
145         cout << "Accelerating to " << speed << " MPH." << endl;
146         systemManagement.insertLog("Braking to " + to_string(speed) + " MPH");
147         break;
148     case 4:
149         int newHeadLightStatus;
150         cout << "Enter new headLight status (0: Off, 1: Low, 2: High): ";

```

```

151         cin >> newHeadlightStatus;
152         if (planningVCS.setheadlights(newHeadlightStatus)) {
153             cout << "Adjusted headLights" << endl;
154             systemManagement.insertLog("HeadLights adjusted successfully");
155         } else {
156             cout << "Invalid input. Failed to adjust headlights." << endl;
157             systemManagement.insertLog("Failed to adjust headlights due to invalid input");
158         }
159         break;
160     case 5:
161         int turnDirection;
162         cout << "(1) Left (2) Right: " << endl;
163         cin >> turnDirection;
164         cout << "Press Enter to Confirm" << endl;
165         cin.ignore();
166         cin.get();
167         if (!planningVCS.setturnSignal(turnDirection - 1)) {
168             cout << "Invalid turn signal." << endl;
169             systemManagement.insertLog("Failed to update turn signal");
170         } else {
171             systemManagement.insertLog("Turn signal changed.");
172             if (planningVCS.attemptLaneChange()) {
173                 cout << "Successfully changed lanes." << endl;
174                 systemManagement.insertLog("Changed lanes successfully");
175             } else {
176                 cout << "Could not change lanes." << endl;
177                 systemManagement.insertLog("Failed to change lanes");
178             }
179         }
180         planningVCS.setturnSignal(-1);
181         break;
182     case 6:
183         int temp;
184         cout << "Enter desired temperature (50-80): ";
185         cin >> temp;
186         planningVCS.setTemperature(temp);
187         systemManagement.insertLog("Temperature set to " + to_string(temp) + " degrees Fahrenheit");
188         break;
189
190     case 7:
191         planningVCS.toggleHazardLights();
192         cout << "Hazard lights turned " << (planningVCS.getHazardLightsStatus() ? "on" : "off") << "." << endl;
193         systemManagement.insertLog("Hazard lights toggled");
194         break;
195     case 8:
196         int temp2;
197         cout << "Press: (0) park, (1) drive, (2) neutral, (3) reverse" << endl;
198         cin >> temp2;
199         if (temp2 == 0) {
200             if (sensorFusion.getCurrentSpeed() != 0) cout << "Vehicle must be stopped." << endl;

```

```

201         else planningVCS.setgear(0);
202     }
203     else if(temp2 == 1){
204         planningVCS.setgear(1);
205     }
206     else if(temp2 == 2){
207         planningVCS.setgear(2);
208     }
209     else if (temp2 ==3) {
210         if(sensorFusion.getcurrentSpeed()!=0) cout<<"Vehicle must be stopped."<<endl;
211         else planningVCS.setgear(3);
212     }
213     else{
214         cout<<"Invalid Code"<<endl;
215     }
216     break;
217 default:
218     cout << "Invalid action code." << endl;
219     break;
220 }
221 break;
222 case 4:
223     tmp = systemManagement.getStatus();
224     if (tmp.empty()) {
225         cout << "The user is not a technician." << endl;
226         systemManagement.insertLog("Invalid status request or the user is not a technician");
227     } else {
228         cout << tmp << endl;
229         systemManagement.insertLog("Status accessed successfully");
230     }
231     break;
232 case 5:
233     display.interactiveDisplay(sensorFusion);
234     break;
235 case 6:
236     cout << "Logging out" << endl;
237     systemManagement.insertLog("User logged out");
238     dataFile.close();
239
240     return 0; // Exits the application
241 default:
242     cout << "Invalid choice" << endl;
243     break;
244 }
245
246 }
247
248
249     return 0;
250 }

```

7. Testing

7.1 Scenario-Based Testing

7.1.1 Sensors Detect an Emergency break

Input:

- Car is driving over 100 miles per hour

Output:

- Car is slowed previous speed
- Driver is issued a warning

Desired Output:

- Car is slowed previous speed

Passed

7.1.2 Driver Activates Automatic Lane Change While in Cruise Control

Input:

- Car is in the middle lane (1 lane on left, 1 lane on right)
- Cruise control is active
- Left turning signal is activated

Output:

- Console: "Successfully changed lanes."
- Vehicle Status: "Current Lane: 0"

Desired Output:

- Console: "Successfully changed lanes."
- Vehicle Status: "Current Lane: 0"

PASSED

7.1.3 Driver Engages Brakes While in Cruise Control

Input:

- Current speed is 60 MPH
- Cruise control is on
- Driver inputs command to brake to 50 MPH

Output:

- Console:
"Cruise control turned off.
Braking to 50 MPH."
- Logs:
"Cruise control turned off due to manual braking
Braking to 50.000000 MPH"
- Vehicle Status: "Cruise Control: Off"

Desired Output:

- Console:
"Cruise control turned off.
Braking to 50 MPH."
- Logs:
"Cruise control turned off due to manual braking
Braking to 50.000000 MPH"
- Vehicle Status: "Cruise Control: Off"

PASSED

7.1.4 Light Sensor Activates Headlights

Input:

- Light level is 3 (High beam threshold by default is 5)

- Headlights are off

Output:

- Vehicle Status: "Headlights: High Beams"

Desired Output:

- Vehicle Status: "Headlights: High Beams"

PASSED

Input:

- Light level is 8 (Low beam threshold by default is 10)
- Headlights are off

Output:

- Vehicle Status: "Headlights: Low Beams"

Desired Output:

- Vehicle Status: "Headlights: Low Beams"

PASSED

Input:

- Light level is 15
- Low beams are on

Output:

- Vehicle Status: "Headlights: Off"

Desired Output:

- Vehicle Status: "Headlights: Off"

PASSED

7.1.5 Technician Requests Log Files

Input:

- Log store is nonempty
- User is a technician
- User requests log files

Output:

- Console: **List of Log Entries**
- Logs: "Logs accessed successfully"

Desired Output:

- Console: **List of Log Entries**
- Logs: "Logs accessed successfully"

PASSED

Input:

- Log store is nonempty
- User is not a technician
- User requests log files

Output:

- Console: "No logs available or the user is not a technician."
- Logs: "Invalid log request or the user is not a technician"

Desired Output:

- Console: "No logs available or the user is not a technician."
- Logs: "Invalid log request or the user is not a technician"

PASSED

Input:

- Log store is empty
- User requests log files

Output:

- Console: "No logs available or the user is not a technician."
- Logs: "Invalid log request or the user is not a technician"

Desired Output:

- Console: "No logs available or the user is not a technician."
- Logs: "Invalid log request or the user is not a technician"

PASSED

7.2 Validation Testing

7.2.1 Adaptive Braking

Satisfied by use case 1 (see 7.1.1 Sensors Detect an Object in the Path of the Vehicle)

7.2.2 Automatic Lane Changing

Satisfied by use case 2 (see 7.1.2 Driver Activates Automatic Lane Change While in Cruise Control)

7.2.3 Cruise Control

Input:

- Cruise control is off
- Current speed is 60 MPH

Output:

- Console: "Cruise Control toggled to ON"
- Vehicle Status: "Cruise Control: On"
- Logs: "Cruise control successfully toggled."

Desired Output:

- Console: "Cruise Control toggled to ON"
- Vehicle Status: "Cruise Control: On"
- Logs: "Cruise control successfully toggled."

PASSED

Input:

- Cruise control is off
- Current speed is 30 MPH

Output:

- Console: "Cannot toggle cruise control when the vehicle is not moving slower than 50mph."
- Vehicle Status: "Cruise Control: Off"
- Logs: "Cruise control cannot be toggled when the vehicle is not moving fast enough"

Desired Output:

- Console: "Cannot toggle cruise control when the vehicle is not moving slower than 50mph."
- Vehicle Status: "Cruise Control: Off"
- Logs: "Cruise control cannot be toggled when the vehicle is not moving fast enough."

PASSED

7.2.4 Dangerous Merge Prevention

Input:

- Car is in the middle lane (1 lane on left, 1 lane on right)
- Left lane is occupied
- Driver attempts to change to the left lane

Output:

- Console: "Could not change lanes."
- Vehicle Status: "Current Lane: 1"
- Logs: "Failed to change lanes"

Desired Output:

- Console: "Successfully changed lanes."
- Vehicle Status: "Current Lane: 1"
- Logs: "Failed to change lanes"

PASSED

7.2.5 Hazard Lights

Input:

- User or technician is logged in
- User toggles to perform action
- User toggles hazard lights to turn them on

Output:

- The hazard lights toggle
- The lights status is shown in the user display

Desired Output:

- The hazard lights toggle
- The lights status is shown in the user display

PASSED

7.2.6 Automatic Light System Activation

Satisfied by use case 4 (see [7.1.4 Light Sensor Activates Headlights](#))

7.28 Automatic Windshield Wiper Activation

Input:

- Rain amount is 0.3 (Wiper threshold is by default 0.1)
- Wipers are off

Output:

- Vehicle Status: "Wipers: Auto"

Desired Output:

- Vehicle Status: "Wipers: Auto"

PASSED

Input:

- Rain amount is 0
- Wipers are on auto

Output:

- Vehicle Status: "Wipers: Off"

Desired Output:

- Vehicle Status: "Wipers: Off"

PASSED

7.2.9 Temperature control

Input:

- User action
- Set temperature
- Set temperature to 65

Output:

- User display prints 65 degrees fahrenheit

Desired Output:

- User display prints 65 degrees fahrenheit

PASSED

7.2.10 Driver Input to Override Autonomous Driving System

Satisfied by use case 3 (see [7.1.3 Driver Engages Brakes While in Cruise Control](#))